



Autonomous Workflow Orchestration System (AWOS) Using Specialized AI Agents

Final Project Report
(BSc Computer Science)

Author: Deya Alnasser
Supervisor: Hector Menendez Benito
Student ID: 21213673

April 22, 2026

Abstract

Manual document processing is costly, inconsistent, and difficult to scale, making it a strong candidate for automation. Existing multi-agent frameworks address this through autonomous coordination, but tend to be complex, expensive, and difficult to audit, limiting their suitability for lightweight deployments. This project introduces the Autonomous Workflow Orchestration System (AWOS), a three-tier multi-agent system for document summarisation, classification, and data extraction, built on directed orchestration principles and deployed entirely on free-tier API infrastructure. Evaluated across 450 documents and 50 routing and refusal test cases, AWOS achieved routing accuracy of 96.7%, classification precision of 95.0%, and a boundary handling logical mean of 94.0%. Extraction precision reached 85.0%, falling short of the 90% target due to failures in structurally complex documents. A hybrid Gemini and GPT-OSS model strategy was empirically validated using an Efficiency Score metric that normalises reasoning accuracy against token consumption, with each model outperforming the other on a distinct agent workload. The results provide evidence that directed multi-agent orchestration on free-tier infrastructure is a viable alternative to autonomous frameworks for bounded document processing tasks, trading a narrower problem scope for substantially lower operational cost and higher auditability.

Originality Avowal

I verify that I am the sole author of this report, except where explicitly stated to the contrary.
I grant the right to King's College London to make paper and electronic copies of the submitted work for purposes of marking, plagiarism detection and archival, and to upload a copy of the work to Turnitin or another trusted plagiarism detection service. I confirm this report does not exceed 25,000 words.

Deya Alnasser

April 22, 2026

Acknowledgements

I would like to express my heartfelt gratitude to my family, especially my grandmothers Mrs. Mariam and Mrs. Najma, my parents Mr. Yosef and Mrs. Amal, and my brothers and sister for their endless love and support. I also thank my friends for their encouragement throughout this journey.

My sincere thanks go to King's College London and the Informatics Department, particularly my supervisor Dr. Hector Menendez Benito, for his invaluable guidance. I am also deeply grateful to Terranova Software, represented by Mr. Alessandro Mella, for their mentorship and support.

Contents

1	Introduction	3
1.1	Background and Motivation	3
1.2	Aims and Objectives	3
1.3	Methodology	4
1.4	Key Results	5
1.5	Contributions	5
1.6	Thesis Structure	6
2	Literature Review	7
2.1	Large Language Models (LLMs)	7
2.2	Agentic AI	12
2.3	Agentic Orchestration Methods	16
2.4	Model Context Protocol (MCP)	20
2.5	LLMs Performance Analysis	21
2.6	Related Work	23
3	Design & Specification	25
3.1	Use Cases	25
3.2	System Requirements	28
3.3	System Specifications	31
3.4	System Design and Architecture	34
4	Implementation	39
4.1	Development Methodology	39
4.2	Technology Stack	40
4.3	System Implementation	43
4.4	Technical Challenges and Implementation Strategies	50
4.5	Testing and Validation	51
4.6	Deployment and Reproducibility	53
4.7	Summary	54
5	Evaluation	55
5.1	Research Questions (RQ)	55
5.2	Experimental Setup	56
5.3	Results	58

5.4	Discussion	60
5.5	Summary	68
6	Legal, Social, Ethical and Professional Issues	69
6.1	Legal Considerations	69
6.2	Ethical Considerations	71
6.3	Social Impact	71
6.4	Professional Considerations	72
7	Conclusion and Future Work	74
7.1	Conclusion	74
7.2	Future Work	75
	Bibliography	82

Chapter 1

Introduction

1.1 Background and Motivation

Manual document processing remains widespread across corporate environments, including invoice management, contract review, insurance claims, and technical reporting. It is costly, slow, produces inconsistent results, and does not scale as document volumes grow, making it a strong candidate for automation. Recent advances in Large Language Models (LLMs) have made this automation feasible. Hsu et al. demonstrated that LLMs can extract data from clinical documents with accuracy exceeding 92% [22], and Hassani et al. showed that GPT-4o classifies dense regulatory text significantly better than earlier models [19]. Together, these results suggest that LLM-based document processing is approaching the reliability required for production workflows.

The prevailing direction in multi-agent AI research, however, focuses on autonomous, self-directing frameworks [3, 31]. While powerful, these systems tend to be complex to operate, expensive to run, and difficult to audit, making them poorly suited to small-scale or cost-constrained deployments where transparency and reproducibility matter. This creates an opportunity for a lighter, directed approach to multi-agent design that can reliably process documents without requiring full autonomous coordination.

1.2 Aims and Objectives

The aim of this project is to determine whether a directed multi-agent orchestration design can deliver reliable document processing on cost-constrained infrastructure, as a viable alternative

to the autonomous frameworks that currently dominate multi-agent research.

To achieve this, the **Autonomous Workflow Orchestration System (AWOS)** was designed, implemented, and evaluated as a three-tier multi-agent document processing system grounded in directed orchestration. The specific objectives are:

1. Design a hierarchical architecture in which an Orchestration Agent routes work to three specialised agents responsible for document summarisation, classification, and data extraction.
2. Support multiple common file formats, including PDF, DOCX, TXT, CSV, XLS, and XLSX, within a single processing pipeline.
3. Develop and validate a Multi-Level Task Refusal mechanism that rejects out-of-scope requests at two levels: the Orchestration Agent and each individual specialised agent.
4. Deploy the system entirely on free-tier API infrastructure to provide empirical evidence that the approach is viable without enterprise-level spending.
5. Evaluate performance against a defined set of non-functional requirements covering routing accuracy, classification precision, extraction precision, and response time.
6. Compare candidate model architectures empirically using an Efficiency Score metric that normalises reasoning accuracy against token consumption to identify the best trade-off between the two.

1.3 Methodology

Development followed a phased, iterative approach to accommodate the stochastic behaviour of LLM-based components, with each phase informing the next. The final system is a single n8n workflow organised into six distinct regions, deployed via Docker Compose [23] and backed by PostgreSQL [43] to support execution auditability. Inference is split between Google AI Studio [17] and the Groq LPU [18], a hybrid choice that balances context window capacity against raw throughput. The Model Context Protocol (MCP) was also evaluated as an alternative coordination mechanism but was dropped after exhibiting excessive token consumption and unreliable routing behaviour, reinforcing the case for the directed approach adopted throughout.

1.4 Key Results

Across 450 documents and 50 routing and refusal test cases, AWOS met three of its four primary non-functional requirements. Routing accuracy reached 96.7% against a target of $\geq 95\%$, classification precision reached 95.0% against $> 85\%$, and boundary handling achieved a system logical mean of 94.0%. Extraction precision fell short of its 90% target at 85.0%, with most failures concentrated in documents with complex layouts and structure. At the model level, Gemini led on throughput during high-volume tasks while GPT-OSS achieved stronger classification accuracy, supporting the hybrid model strategy over commitment to a single provider.

1.5 Contributions

The main contributions of this work are as follows:

- **System:** a fully implemented, containerised multi-agent document processing system that handles five file formats across three specialised agents and deploys on free-tier infrastructure through a single initialisation script.
- **Architecture:** a directed orchestration design that achieved 96.7% routing accuracy without autonomous self-direction, demonstrating that dependable multi-agent coordination does not require Planner-Executor-Evaluator-Judge or MCP-based frameworks.
- **Refusal mechanism:** a Multi-Level Task Refusal pattern operating at both the orchestration and agent layers, achieving 100% accuracy at Level 1 and 80% at Level 2 in the boundary handling evaluation.
- **Evaluation metric:** an Efficiency Score that normalises reasoning accuracy against token consumption, providing a model comparison measure independent of variable network latency.
- **Empirical validation:** a comparative evaluation of three model architectures across 450 documents, providing evidence that free-tier Gemini models can handle high-volume document processing without paid enterprise access.

1.6 Thesis Structure

This thesis is organised as follows. Chapter 2 reviews literature on LLMs, agentic AI, agentic orchestration methods, and related work. Chapter 3 sets out the system design and formal requirements. Chapter 4 details the implementation of each component. Chapter 5 presents the evaluation and results. Chapter 6 examines the legal, social, ethical, and professional implications. Chapter 7 closes with findings, limitations, and future work.

Chapter 2

Literature Review

This literature review develops both the theoretical and practical foundation for the project through an examination of those areas that form the basis of modern Autonomous Workflow Orchestration Systems. This chapter begins with the background and history of Large Language Models and their increasing use in industry. It continues on to introduce the Agentic AI paradigm, its operational mechanisms and relationship with LLMs. Then, the literature review examines agentic orchestration techniques specifically focusing upon hierarchical coordination as a method to address the shortcomings of individual LLMs. Next, it assesses the capability of the Model Context Protocol as a potential coordination standard alongside its current ecosystem constraints. Subsequently, the literature review conducts a comparative evaluation of existing LLMs to assist in selecting models best suited for performing specialised functions. Finally, the literature review surveys related work to place the project within the broader body of research and identify the specific gap addressed by the project.

2.1 Large Language Models (LLMs)

Advances in Large Language Models (LLMs) represent a fundamental paradigm shift in artificial intelligence, marking a transition from special-purpose language tools into comprehensive reasoning tools. This large-scale development of these models was made possible due to the extensive scale of LLMs, often having hundreds of billions of parameters; as well as their base within the transformer architecture [54]. LLMs acquire broad world knowledge through extensive self-supervised learning, which enables them to generate text that is both fluent and contextually informed [54]. These developments in LLMs represent significant increases in data,

computing power, and model size than prior pre-trained language models. Consequently, LLMs have enabled new functionalities, including multi-step reasoning, in-context learning, and task generalisation [25, 54]. Beyond model design, modern LLMs also utilise methods to align the model’s behaviour with human intent, including instruction tuning and Reinforcement Learning from Human Feedback (RLHF), and therefore can accomplish a wide range of tasks without being specifically trained on those tasks [25]. As a result, LLMs now function as versatile language systems, that support a number of applications, including classification, summarisation, interactive reasoning, and decision-support workflows [25, 54].

2.1.1 Evolution of LLMs

The evolution of LLMs resulted directly from the development of Natural Language Processing (NLP), which evolved over time from rule-based systems to statistical learning to neural architectures, capable of learning language patterns [54]. Prior to the maturation of transformer architecture, early neural models including Recurrent Neural Networks and Long Short-Term Memory provided an improved sequential text modelling; however, these models had a significant problem with the strictly sequential nature of how they processed texts. This design caused significant problems in terms of how well the model could handle long-range dependencies in the text, and how efficiently the model could perform parallel computations [47]. A major innovation in the area of NLP was introduced with the transformer model. Using self-attention instead of recurrence, the model could process all tokens at once and selectively focus on relevant contextual information [47]. This innovation significantly increased the performance of the model, its scalability, and its training efficiency. The transformer model created a foundation for building modern large-scale language models.

Using the transformer model as a base, the area of NLP experienced rapid progress with the development of very large pretrained language models. A landmark example in this era was BERT. BERT demonstrated the potential of using large amounts of bidirectional training data to produce models with high performance in a variety of subsequent tasks. Building upon the fine-tuning paradigm introduced by earlier models like OpenAI GPT, BERT showed that using deeply bidirectional representations allowed lightweight task-specific models to achieve state-of-the-art results during fine-tuning [13]. Subsequently, models such as GPT-3 demonstrated that by increasing both the size of the parameter space and the amount of training data used, it was possible to create models that exhibited emergent properties, including the ability to perform a few-shot and zero-shot learning that did not exist in smaller models [8]. The trend

of scaling continues to be a key aspect of the development of LLMs today. As models continue to grow in terms of parameters, training data, and computing resources used, there will likely be improvements in the models’ ability to generalise and reason about the world [54]. The current evolutionary phase is focused on alignment, which refers to the practice of building models that provide helpful responses to human requests while maintaining safety. Alignment is achieved primarily through RLHF, where human feedback serves as a reward signal for fine-tuning models to follow human-written instructions [33]. Ouyang et al. have also shown that this process can be very efficient; For instance, the authors report that the 1.3-billion parameter InstructGPT model outperformed 175-billion parameter GPT-3 base model, despite having 100 times fewer parameters [33].

2.1.2 LLMs Adoption in the Industry

The large-scale application of LLMs has taken off rapidly throughout a number of organisations and companies to help increase automation in knowledge-intensive roles and create better decision-making processes. A recent survey indicated that LLMs were being used in various types of industries including finance, healthcare, law firms, education, customer service, and software developers for a variety of purposes to include generating text; analysing data; creating code; and facilitating customer interaction [10]. Empirical research by Noy and Zhang indicates that professional writers have the potential to realise a considerable amount of increased productivity when they use AI [32]. The data collected in the experimental study demonstrated that professionals’ use of ChatGPT resulted in a statistically significant average decrease of 11 minutes in completion time per task, as well as increased user job satisfaction by approximately 0.47 standard deviation units[32]. Interestingly, the study found that generative AI acted as an “equalising effect”; the quality improvements were significantly greater for less-skilled writers than for highly-skilled ones, effectively compressing the productivity distribution and decreasing inequality between workers [32].

However, as LLM based agents begin to be integrated into the complex information systems found in larger enterprises, research and evaluation have indicated that the current LLM models have difficulty replicating realistic multi-step workflows common in enterprise settings. Industrial studies show that despite their performance in the EnterpriseBench Benchmark, for example, the best performing agent using an LLM achieved a task completion rate of only approximately 41.8% across different domains such as software engineering, HR, Finance and Administrative roles [50]. Furthermore, many companies are constrained from adopting LLMs

on a large scale due to high computational needs, concerns about company data privacy and governance policies, as well as problems related to incorporating LLMs into existing workflow and IT infrastructures [10].

Overall, these findings indicate that reliable deployment of LLMs within complex business environments requires structured coordination mechanisms and systematic design. For this project, these industrial limitations, particularly the 41.8% task completion rate and integration challenges motivated the development of a coordinated multi-agent architecture to ensure the deterministic workflow reliability that standalone LLM deployments cannot provide.

2.1.3 Core Capabilities to Document Workflows

Document-processing workflows are supported by various characteristics that make LLMs suitable for such tasks. LLMs can generalise across tasks by prompt-based interaction, allowing them to classify, summarise, or extract information without parameter update or particular task retraining. Liu et al. describe this as a critical aspect of in-context learning, where a model learns a new task only from examples or descriptions given within the prompt [29].

Semantically, LLMs are highly proficient at understanding the fine-grained meaning, identifying relationships between different concepts, and deriving the author’s intention. These abilities are important aspects of document-routing, extracting information from documents, and interpreting ambiguous content [13, 59]. Information retrieval research has shown that these models have been able to identify more complex context and semantic patterns than previous neural-based approaches [59].

Additionally, LLMs support transforming unstructured document data into actionable output. For example, LLMs can create summaries of documents, answer questions regarding multiple sections of a document, and extract structured information from unstructured documents. The combination of retrieval, interpretation, and generation enables them to produce transformed output that supports business decisions and automates workflows [59]. These core LLM capabilities, in-context learning, semantic understanding, and format-agnostic processing, form the foundation for the specialised agents in this project. By utilising the LLM’s native semantic capabilities through prompt engineering, the agents can autonomously interpret and process documents regardless of type or format, without requiring custom programmatical logic or task-specific model training for each document format.

2.1.4 LLMs Limitations in Workflows

Despite their strong language-generation capabilities, LLMs exhibit several limitations that directly affect their reliability in document-processing workflows. Hallucinations, which occur when models produce content that is natural and dynamic but genuinely inaccurate, unfaithful to the source, or unverifiable, remain a serious concern. Ji et al. differentiate between intrinsic hallucinations; where output contradicts the source, and extrinsic hallucinations; where models introduce information not grounded in the input [24]. These behaviours arise from architectural and training-related factors, including misallocation of attention, decoder errors during generation, and the bias of models to prioritise internal parametric knowledge over knowledge grounded in the sources themselves [24]. Such properties reduce the trustworthiness of LLMs when workflows require factual precision and consistent reasoning over documents.

In addition to the issue of hallucination, LLMs are also limited structurally to perform multi-step or structured reasoning. Wang et al. showed that although LLMs can accomplish multiple independent classification tasks, the performance of the model is significantly reduced when tasks require the combination of classification with index selection, even under conditions of zero-shot, few-shot, or chain-of-thought prompts [53]. The frequency of failure across various models indicates a larger capability gap and shows that LLMs do not understand task variations in a way that humans would [53]. These structural limitations present problems for the integration of LLMs into complex, multi-process enterprise workflows.

2.1.5 Summary

To summarise, although this discussion has confirmed the presence of the necessary semantic capabilities within LLMs for processing documents, they have a number of serious limitations relative to both multi-step reasoning and fact-based reliability that will hinder the application of LLMs for automation of enterprises. Complex workflows and the likelihood of intrinsic hallucinations show a capability gap between how an individual LLM can handle documents; therefore, a single monolithic model will not be suitable to reliably manage documents. The primary reason this project chooses to adopt a multi-agent orchestration architecture, rather than enhancing each LLM individually to improve their performance, is to utilise the model’s ability to generalise and understand semantic relationships, while using external mechanisms to structure its reasoning process and thereby reduce the internal reliability issues of LLMs.

2.2 Agentic AI

Historically, automation has focused on relieving human drudgery and optimising business operations, evolving from the physical mechanisation of the Industrial Revolution to the automation of repetitive mental tasks and information-processing during the Information Revolution [4]. These earlier systems primarily automated routine, predictable work tasks with well-defined solution spaces. However, as a result of increasing industry demand for autonomous decision-making capabilities in complex operational environments, automation technology is experiencing a major transition away from this traditional model to an emerging paradigm referred to as Agentic AI [4]. Different from previous generations of automation systems, which were capable of automating discrete, independent process steps, Agentic AI systems connect discrete independent steps into a coordinated end-to-end process, such as claim adjudication and research workflows that can track progress and recover from errors automatically, without human intervention [5].

Agentic AI is defined by its self-directed and goal-oriented intelligence, which separates it from prior generation generative models that were simply responsive to user prompts; unlike traditional LLMs, Agentic AI systems are proactive in identifying sub-goals, selecting tools, and executing multi-step actions with minimal human oversight [5]. The mechanism underlying Agentic AI involves continuous perception and strategy adaptation within a goal-directed loop where the agent perceives its environment and adapts its strategy to achieve its goals until clearly defined termination criteria are met [7]. Additionally, Agentic AI agents are able to autonomously access memory and execute decisions through natural language; therefore, they do not require constant permission from operators to perform each individual task [4].

2.2.1 Agentic AI Relationship with LLMs

Central to this capability is the role of LLMs, which provide the underlying reasoning architecture. While Agentic AI uses LLMs, they are used to support the foundational cognitive architecture and do not operate independently. In contemporary architectures, LLMs represent the primary reasoning and perception functions of agents that allow them to parse user queries, develop multi-step solutions and provide intelligent interactions with the environment [38]. Similarly, conceptually a foundation model is to generative AI as an agent is to agentic AI; agentic AI extends foundation models with the capability to use tools, access memories and engage in reinforcement learning [40].

The relationship between agentic AI and generative AI provides significant advantages over

simply using LLMs to generate text, including mechanisms such as ReAct (Reasoning and Acting), a framework that enables LLMs to interleave reasoning traces with task-specific actions [57]. The ability to dynamically induce, track and update action plans enables the model to react to exception-based scenarios, while providing a mechanism to acquire information from external sources through actions [57]. Moreover, by generating verbal reasoning traces along with generated actions, the model has the ability to perform dynamic reasoning to generate and maintain high-level plans and is not limited to reacting to input [57].

The combination of generative and agentic AI solves the execution limits of generative AI by itself. Generative AI centres on producing content from a specific prompt. In contrast, agentic AI uses aspects of reinforcement learning to manage entire workflow processes [38, 40]. Through sequential interactions with the environment and feedback received that guides future steps, agentic AI converts the static reasoning provided by LLMs into active and multi-step problem-solving to manage all possible tools and environments [40].

2.2.2 Agentic AI for Document Processing

Historically, traditional AI was limited by strict rules and narrow operations like predicting, classifying, and retrieving documents. Agentic AI provides an avenue to move beyond these restrictions through the incorporation of human-like intelligence to accomplish multifaceted, multi-stage tasks, such as document analysis, legal reviews and strategic decision-making [46]. By incorporating these agentic capabilities into their operational workflows, organizations can significantly reduce the need for manual intervention while improving their use of resources and decision-making efficiency. This flexible adaptability to complexities, such as those found in many legal workflow environments, makes Agentic AI a critical component in the development of modern workflows and increased financial efficiency [46].

A key limitation to the application of LLM-based agents in these same legal workflows is the inefficient information retrieval process resulting from the agent’s tendency to produce hallucinations, experience context drift and provide inconsistent responses [58]. In contrast to traditional retrieval methods, agentic frameworks are based on generative information retrieval to dynamically generate and combine multi-sourced knowledge, thus allowing memory-augmented, context-sensitive reasoning [58]. These frameworks incorporate structured reasoning and self-reflection or evaluation to ensure high retrieval accuracy and consistent decision making [58]. The positive effects of these mechanisms can be quantified; test results indicate a 42.6% cost savings and a 60.8% increase in document processing speed when compared to baseline AI

systems, with scale testing demonstrating that they can manage high volume query loads [46]. Additionally, it is anticipated that this technology will help improve data quality and compliance by automating mundane tasks and minimizing errors, thereby freeing employee time to focus on more valuable tasks [52].

Beyond individual document tasks, scaling these agentic approaches across organizations requires coordination structures between agents and humans. Although humans might represent a delay to the completion of complex autonomous operations, researchers have been seeking scalable solutions to facilitate collaborative autonomous communication between agents [28], through the use of frameworks such as role playing, which enables agents to complete tasks by using initial prompts to ensure that their actions align with those intended by humans and provides insight into how agents view cognition when working together [28]. To ensure successful operation of these autonomous entities, a significant management structure will be required. Agentic Business Process Management (ABPM) represents the application of software agents to accomplish organizational goals using agent-based abstraction for both design and analysis [52]. Therefore, organizations must clearly define business objectives, establish legal and ethical bounds, and create safety procedures for the integration of humans and agents, including risk assessment for the collaboration of humans and agents, to ensure the responsible deployment of ABPM [52].

This project uses a combination of agentic capability approaches for an agent to orchestrate its own actions by utilising a hybrid agent architecture. The Orchestration Agent analyses the user’s intent through prompt classification and routes tasks to the appropriate document processing agents (Summarisation, Classification and Data Extraction). Each specialised agent executes its assigned task independently using LLM reasoning capabilities.

2.2.3 Agentic AI Limitations

There is an inherent risk to the Agentic AI paradigm; primarily due to goal misalignment and reward hacking. Because agentic systems will relentlessly pursue optimisation objectives based on specifications, even minimal ambiguity can result in unforeseen or unsafe consequences, often referred to as the specification problem [34]. In addition to simply misinterpreting instructions, agents may also engage in reward hacking by optimising vague or inaccurate representations of value or adopting means-to-an-end goals that are reckless to pursue a proper objective [11]. As such, there exists an overarching risk that agents inherit problematic characteristics in the reward functions of the domains in which they operate [11].

The risks present in single-agent systems are magnified when operating in Multi-Agent Systems. Collective interactions among agents produce Emergent Behaviour patterns of coordination and feedback loops that were not anticipated by developers, but instead result from the interactions of the individual agents [34]. Emergence has the ability to enhance efficiency of operation in Multi-Agent Systems; however, it inherently adds complexity through the generation of unpredictable and unstable global dynamics [34]. In essence, the autonomous nature of emergent behaviours of these agents generates significant responsibility gaps where negative events have occurred, yet no particular agent can be identified as responsible [11]. The liability mechanisms currently available to govern the actions of humans are incapable of managing systems that combine the attributes of both tools and actors [34].

Finally, the design of agentic systems creates numerous security and systemic vulnerability issues. There exists an enormous attack surface with the potential for prompt injection and policy hijacking, as a result of both the autonomous nature and the interconnectedness of these systems [34]. The self-authorization loop presents an increased risk that once a malicious agent has taken control, it will be able to propagate its malicious effects throughout a system without human intervention [34]. On a broader macroscopic scale, the introduction of these types of autonomous actors has the potential to disrupt social structures designed for human interaction, resulting in systemic and delayed impacts as more critical societal functions are performed by autonomous entities [11].

This project addresses these risks with architectural constraints. Directed orchestration aligns agent goals by constraining autonomous actions to predefined scopes, preventing specification problems. Centralised orchestration eliminates emergent behaviour by preventing direct agent-to-agent communication. Explicit task refusal patterns maintain human oversight by rejecting out-of-scope requests rather than executing them autonomously.

2.2.4 Summary

In summary, Agentic AI creates goal-oriented autonomous systems that extend beyond reactive LLM-based interaction. The risk of goal misalignment, emergent behaviour, and the potential for security vulnerability as discussed above will require that there be established forms of formalised coordination for enterprises to use Agentic AI.

This project develops a hybrid autonomous document processing system utilising agentic capabilities (i.e. semantic reasoning, selecting tools) for summarisation, classification, and data extraction across diverse document formats, while maintaining deterministic execution

and auditability through directed orchestration. This approach addresses the gap between autonomous agents’ flexibility and the operational reliability required for professional document workflows.

2.3 Agentic Orchestration Methods

Agentic orchestration methods have been increasingly utilised to compose multiple specialised agents to collaboratively accomplish complex tasks [55]. However, while foundational frameworks facilitate multi-agent conversation, they often lack mechanisms for dynamic, value-driven routing. To address this, a primary objective of the orchestration method proposed in this project is to select the best agent for a specific task to produce the maximum amount of value using offline data. Specifically, this workflow seeks to find those LLM configurations that are the most suitable, feasible, and cost-effective for assignment. Structured coordination is also needed here because having many agents available does not automatically create benefits. Indeed, unstructured interactions lacking explicit control flow often fail to improve overall accuracy of the system or optimise its effectiveness [55].

Orchestration is designed to address the shortcomings of LLMs by allowing centralised control over workflow processes to help solve the context window problem that results in both coherence and contextual relevance of generated responses decreasing as the interaction with the user extends beyond a specific amount of time of interaction [45]. Additionally, it provides a solution to the inability of humans to efficiently decide when to outsource tasks to agents; users often fail to recognise when and how to utilise automation tools for support and therefore function poorly as an orchestrator without some form of automated guidance [55].

2.3.1 Orchestration Architecture and Workflow

At its core, the architecture of an agentic orchestration workflow is an intelligent routing mechanism designed to provide maximum accuracy [55]. Modern frameworks manage the complexity of multi-agent collaborations by decomposing objectives into specific actionable procedures assigned to specialised roles, often functioning as an assembly line [20]. While these Standardised Operating Procedures ensure tasks are completed correctly, they are typically rigid. To build upon this, the orchestration engine developed in this project introduces dynamic adaptability; when it receives a series of dataset input streams, it primarily determines the best agent for each particular task in real-time, ensuring optimal execution and accuracy. Modern orchestra-

tion goes beyond simple routing to allow for complex collaboration. After gathering the needed data and context, the orchestration engine coordinates a multi-LLM response where multiple models work together to produce a comprehensive response utilising their respective strengths [45, 55].

To execute this complex collaboration, frameworks build upon the aforementioned role decomposition [20] by structuring workflows into strict hierarchies. This hierarchical design is formalised by Tokal et al. into four fundamental components: **Planner**, **Executor**, **Evaluator**, and **Judge** [45]. In the outlined structure, the Planner is considered the architect responsible for developing a plan to accomplish the objective of the current stage using the available resources or tools [45]. The Executor then performs the steps identified by the planner in an iterative fashion [45].

Furthermore, this hierarchical approach to orchestration includes mechanisms for providing quality control via iterative recursive loops. As stated previously, consistency within the framework is achieved by reflection loops. Specifically, as soon as an output is generated from executing a given set of tasks, the system immediately checks whether the output was satisfactory. If the output was deemed unsatisfactory, the system then iteratively evaluates or reuses existing data sources until satisfactory results are obtained [20, 41]. While evaluation occurs at a lower level with regard to verifying individual execution step compliance by the **Evaluator**, it also occurs at a higher order with regard to reviewing stage progression by the **Judge** [45].

However, as Tokal et al. explain, an entirely autonomous self-correcting loop would require substantial overhead. This project utilises the underlying principles by implementing validation logic as a **Judge**, validating the business schema of tasks performed by the specialised agents prior to allowing the workflow to reach its end-state.

2.3.2 Adoption and Benefits of Orchestration

The adoption of orchestration frameworks is primarily because they allow for higher or better performance than monolithic LLMs have historically been able to offer. Empirical studies indicate that orchestration generally produces a better level of precision than an individual LLM, such as GPT-4, producing notably higher results without requiring knowledge of the best performing model for each task [27]. Thus, the overall performance of the system is equivalent to the performance of the most skilled member of the team [20], not limited to the weakest link in the chain of generalist models [12].

In addition to improved accuracy, the use of orchestration has greatly increased the effi-

ciency of computations. The implementation of primitive-level data flow graphs enables the framework to take advantage of parallelism and therefore produce a significant speed-up over previous systems [30]. Additionally, adaptive orchestration techniques improve operational cost by optimising the reasoning path for each process, where the orchestrator learns to terminate the process earlier once sufficient information has been collected, resulting in simultaneous improvement in both the effectiveness of reasoning and the utilisation of resources [12].

Lastly, the use of orchestration provides adaptability and scalability. Unlike monolithic models that face efficiency limits when solving complex problems, the central orchestrator dynamically assigns tasks to specialised agents to enable flexible and evolving forms of collective reasoning [12]. This fine-grain control not only provides a greater number of possible scheduling improvements [30], it also creates adaptive systems that limit hallucinations [41], through the creation of compact cyclic reasoning structures [12].

2.3.3 Orchestration Limitations

Although orchestration architectures have many benefits, there are several structural and operational issues with them, which limit their use in production environments. An overarching issue of concern with respect to orchestration architectures is performance; since many orchestration architectures are cascade-based, latency and redundant computation are introduced into the architecture due to the need for sequential querying of the smaller models prior to the larger models being queried [27]. The operational rigidity of classifier-based routers compounds this issue, since they require retraining to accommodate the inclusion of new models [27]. Deterministic systems also require an explicit manual integration of new capabilities instead of merely modifying the underlying LLM [44]. Additionally, orchestration architectures experience a complexity barrier when it comes to scalability. The inability of current LLMs to orchestrate large scale workflow processes that contain both extensive logical structures, such as loops and branches [15], is a major limitation to the LLMs’ ability to address the complexity of real-world problems [15].

In addition to being limited structurally, there is also a trade-off in terms of generalisation and adaptability for an orchestrator architecture. As deterministic logic is often used to ensure reliability in orchestration architectures, they lose the inherent generative flexibility of LLMs, and therefore cannot be adapted to novel or unforeseen user requests that fall outside of the predetermined paths [44]. Therefore, the greatest constraint of orchestration architectures is their inability to generalise across a wide array of problem scenarios [37]. Although an

orchestration architecture can effectively break down a complex task into smaller components, it does not inherently possess the ability to iterate on learning based on the results of those decompositions; therefore, an orchestration architecture is unable to develop autonomously [37].

This project addresses these limitations through deliberate architectural decisions. It uses a multi-API approach to reduce cascading latencies: lightweight models are used for both orchestration and routing, while larger specialised models are used for longer running tasks that require additional context, limiting the sequential overhead. Reliability is prioritised over generality by implementing deterministic routing. Task refusal based on an explicit definition of scope is also implemented rather than relying on a generic processor to handle all out-of-scope requests.

2.3.4 Summary

Directed orchestration is a constrained orchestration pattern in which all routing decisions and execution paths are defined explicitly at design time, rather than resolved autonomously at runtime, trading generalisation for determinism, auditability, and cost efficiency. This project does not adopt the full Planner-Executor-Evaluator-Judge model because of the significant overhead in cost, time and resources associated with autonomous self-correcting loops; instead, it adopts the fundamental principles from which the framework was developed. Specifically, the project implements validation logic as a judge to verify agent output before the workflow reaches the end-state. This enables reliable and efficient execution, while also minimizing the complexity barrier and reliability constraint issues associated with current orchestration methods. As demonstrated in this section, parallelism enables accurate and computationally efficient results; however, the same body of literature also provides evidence of several architectural risks, such as increasing latency and reducing operational flexibility, associated with using cascade-based systems. In order to achieve a balance between these two goals of enhancing the generative flexibility of LLMs and minimizing hallucinations, the system design of this project emphasises targeted validation over purely deterministic routing. The separation of execution from evaluation within the framework addresses the generalisation gaps often present in rigid orchestration models; thereby providing a validated execution process capable of producing consistent performance across a range of complex, real-world tasks.

2.4 Model Context Protocol (MCP)

2.4.1 MCP Capabilities and Interoperability

The Model Context Protocol (MCP), in its emergence as an integration interface to bridge the gap between LLMs and their external utility components, represents a technical level of abstraction; a typed abstraction layer to describe interactions between LLMs and tool sets. To this end, MCP has the potential to reduce fragmentation between various systems [21]. In addition, MCP enables LLMs to move beyond static prompt-response interactions by providing a universal interface through which agents can access contextual information [56] and can invoke external functions, execute programmatic code, and manipulate digital environments [21]. Furthermore, MCP acts as a semantic complement to traditional API architectures [48]; thus establishing a new paradigm of interoperability [48] enabling structured agent-to-agent interaction and dynamic capability discovery [36]. Lastly, through standardisation of how models interact with data and tools, MCP will enable systems to bridge the cognitive-operational divide by providing connections between autonomous reasoning and governed, deterministic enterprise data; thereby, allowing reasoning to be based on persistent context propagation versus isolated, stateless transactions [48].

2.4.2 MCP Architecture

The architecture of MCP addresses the technical deficiencies in the traditional integration models of REST and GraphQL; both are lightweight web-native paradigms that are stateless by nature and have a fixed schema, thus unable to sustain semantic coherence across autonomous reasoning steps [48]. The addition of a semantic and context-aware communications layer to MCP allows for separating how tools are built from how agents invoked them. This enables the integration of heterogeneous tools into MCP regardless of any single model or agent framework [21]. This creates a modular environment where capabilities are dynamically discovered and negotiated at runtime, eliminating the need for platform-specific adapters [21]. As a result, MCP-based architectures facilitate persistent context propagation across distributed systems, significantly increasing the reuse and scalability of enterprise-ready agentic workflows [48].

2.4.3 MCP Limitations

Although there are theoretical benefits in implementing MCP, the current state of the field with respect to providing effective solutions for security, remote deployment, and automatic discovery

of tools has been largely undeveloped [21]. Incorporating this type of protocol will add considerable architectural complexity to an application and may include the need for specialised infrastructure elements, such as separate context stores and governance pipelines, which can greatly increase computational overhead [42]. Furthermore, recent academic studies have identified specific vulnerabilities linked to inconsistent maintenance standards and unpredictable control flow behaviour of the model [21]. Additionally, open interoperable standards are often met with strategic resistance from established digital platforms that deploy closed ecosystems to maintain proprietary control of their data resources [21]. Lastly, many of the current MCPs currently do not lend themselves well to applications requiring real-time processing of streaming data without substantial additional development[14].

2.4.4 Summary

In summary, MCP is a major step toward standardizing communication between LLMs and external tools, enabling modular integration and dynamic identification of available capabilities. These capabilities made MCP an initial consideration for coordinating the orchestrator and specialised agents within this project. Nevertheless, as outlined in this section, the immaturity of the MCP ecosystem regarding both tool discovery and security, along with the substantial architectural complexity imposed by its infrastructure requirements, adds considerable overhead for a single-user document processing workflow. As such, this project does not adopt MCP; a detailed technical justification for this decision is presented in the Implementation Chapter.

2.5 LLMs Performance Analysis

The ability of the **Autonomous Workflow Orchestration System** (AWOS) to be technically viable depends on selecting appropriate LLMs through empirical benchmarks. The current literature shows that the trend toward better performing models is becoming increasingly apparent. Models that previously were unable to match the expertise of humans in complex textual analysis are now outperforming them in many fields.

2.5.1 Performance Accuracy and Achieving Human Parity

Research performed recently demonstrates that contemporary LLMs are exceeding human baseline performance in the complex analytical functions needed for deep contextual understanding. These new models are also showing themselves to be cost effective and an accurate replacement

for manual coding used by researchers who conduct data analysis. Evaluations comparing the best-in-class models, such as GPT-4 and Claude 3.5 Sonnet, demonstrate that these models have achieved established levels of inter-coder agreement. Unfortunately, outsourced human labor rarely reaches this level of achievement [6]. In specific professional disciplines, such as extracting information from clinical guidelines, GPT-4o and Gemini-1.5-Pro have produced highly reliable results with accuracy rates exceeding 92%, and comparable success scores when compared to human expert labels [22].

2.5.2 Evolutions in Architectures: Advanced versus Legacy Models

Large language model architectures have developed rapidly resulting in significant improvements in performance relative to older models. Specifically, GPT-4o was shown to provide a meaningful improvement over legacy systems such as GPT-3.5-turbo and BERT for classifying complex regulatory language. Precision remained high across all three models when fine-tuned; however, GPT-4o demonstrated a trade-off for identifying relevant document sections under few-shot prompting conditions. Fine-tuning is still considered the gold standard for precision; however, some researchers are using few-shot prompting with newer models to prioritise recall. Recall rates were up to 97% in identifying relevant document sections using a few-shot prompt [19]. Llama 3 and GPT-3.5 produced summaries with greater contextual fidelity and clarity compared to alternatives such as Gemma-2 and Mixtral 8x7B [39]. Although Mixtral may produce stronger numerical results, it does not produce the same level of structural clarity and coherence needed for high quality technical abstracts [39].

2.5.3 Reliability and Task-Specific Reasoning

High-level systemic reliability and output reproduction are essential characteristics of an autonomous orchestration system. Reproducibility and reliability of LLM-based outputs are greatly improved over the inherent variability exhibited by human reviewers [6]. High-level evaluations of GPT-4o and Gemini-1.5-Pro indicate reproducibility rates approaching 99%, providing evidence that both models can be reliably integrated, suggesting these models are suitable candidates for integration into real-time systems [22].

Performance metrics and capabilities are being utilised to select an architectural design for this project that can be applied to application-specific uses. Although research on which of these models has been performed is evolving rapidly, empirical studies have indicated that GPT-4o achieves the highest overall accuracy for classification-based applications [19]. Additionally,

Llama 3 was identified as providing the most organised structure and clarity when it comes to producing accurate technical summaries [39].

2.5.4 Summary

This discussion identified where current LLMs stand in terms of their performance relative to baseline expert performance. This review also demonstrates that several of these models can now achieve equivalent or superior performance when reasoning through complex contexts and extracting highly accurate data. While there exist several benchmark models that represent a range of classification accuracy and summarisation quality; all of the aforementioned models continue to advance at a rapid pace. GPT-4o and Gemini-1.5-Pro were better for the classification and extraction functions based on the highest accuracy and reproducible results. The model Llama 3 was better for the summarisation function because it provides the most structurally clear output. The evaluation conducted in this project compared foundational models against newer open-source equivalents available under free-tier access. The merits and limitations of each model are examined in detail in Chapter 5.

2.6 Related Work

The development of the AWOS is located in a larger trend of transitioning away from monolithic LLM applications, toward more advanced multi-agent-based ecosystems. This section reviews the current state of agentic orchestration technology, focusing on the technical shift toward hierarchical workflow management.

2.6.1 Architectural Paradigms and Agentic AI

As the field of AI moves toward what can be called the neural paradigm of agency, with a focus on moving from internal symbolic logic to prompt-driven orchestration, Abou Ali et al. define this as the emergence of Agentic AI; an architectural strategy that enables the coordination of many specialised agents to resolve problems beyond the capabilities of a single model [1]. Additionally, Adimulam et al. support this advancement of Agentic AI, stating that the next generation of AI consists of these coordinated collections, where the collective behaviour of coordinated agents contributes more to system value than the capabilities of any single member [3].

2.6.2 Scalability and Accuracy

One reason for AWOS to have a hierarchical architecture is based on empirical evidence showing that there exists a significant performance difference when comparing single-agents and multi-agent architectures. Klang et al. show that multi-agent systems maintained substantially higher accuracy than single-agent counterparts under increasing load (90.6% versus 73.1% at 5 tasks; 65.3% versus 16.6% at 80 tasks) [26]. The authors attribute the continued level of accuracy for multi-agent systems to the module isolation of worker agents, preventing context interference that often results in decreased reasoning for single-prompt models [26]. In addition, according to Abou Ali et al. modular architectures achieve scalability through coordinated specialisation rather than relying on the reasoning depth of any individual agent [1].

2.6.3 Agentic Orchestration and Hierarchical Management

Masters et al. identify the “Autonomous Manager Agent” as a central open challenge in coordinating complex multi-agent workflows. Masters et al. suggest that while high-level goal decomposition is possible with modern LLMs, it remains an open challenge to automate task graph construction, which models still struggle to optimise [31]. The managerial layer acts as a centralised coordination layer governing planning and execution, thus preventing logical contradictions or unlimited autonomy [3]. As stated by Adimulam et al., such an orchestration layer is necessary for enterprise reliability enabling independent agents to operate as a unified, purpose-driven system [3]. The structured protocols discussed by Adimulam et al. also allow for auditing in terms of transparency and compliance with regulations for verifying and reproducing automated workflows [3].

2.6.4 Conclusion

Collectively, these studies justify the proposed three-tier architecture and demonstrate that an orchestrator-worker hierarchy provides effective solutions for addressing the limitations of accuracy and scalability associated with LLMs operating in a standalone capacity. Unlike high-complexity enterprise or clinical systems that employ fully autonomous manager agents, AWOS operates within a single-user scope where this overhead is not justified. This justifies the application of directed orchestration as a more proportionate alternative to self-regulating loops, as discussed in Section 2.3. Ultimately, AWOS fills a specific void in the literature by providing a practical framework for developing lightweight, auditable multi-agent systems focused specifically upon professional document processing environments.

Chapter 3

Design & Specification

This chapter provides the formal framework for the Autonomous Workflow Orchestration System (AWOS). It translates the core project objectives into a structured set of Use Cases and System Requirements. Then, the System Specifications defines behavioural specifications for component interoperability. Finally, in the System Design Section, the architectural choices made (i.e. Directed Agentic Orchestration) will be justified to produce a cost-efficient, reliable, and auditable implementation.

3.1 Use Cases

This section describes the functional scope of the AWOS as a collection of use cases. It uses a chat interface so that users can upload documents, and specify their needs for processing those documents using natural language. After a user makes a request, an orchestration agent analyses the request and routes it to one of the three existing specialised agents (Summarisation, Classification, or Data Extraction) for execution.

Document Formats

The system supports cross-document type functionality with respect to the following document formats `.pdf`, `.txt`, `.docx`, `.csv`, `.xls`, `.xlsx`.

3.1.1 UC-01: Document Summarisation

- **Actors:** Business analyst, Manager.
- **Goal:** Provide a concise summary of the insights and content contained within a document.

- **Preconditions:**

- One or more documents have been uploaded in a supported document format.
- Each of the uploaded documents contains digitally readable text.

- **Main Flow:**

1. The user uploads documents and provides a natural language summarisation request.
2. The system reads the raw text contained within the uploaded documents.
3. The Orchestration Agent analyses the prompt's intent and routes the user's input to the Summarisation Agent.
4. The Summarisation Agent performs the requested task, and creates a concise abstract of the document.
5. The system displays the document summaries to the user.

- **Alternative Flows:**

- **Multiple Document Processing (AF-1A):** if a user has uploaded multiple documents, the system processes each document individually, and then creates a summary for each.
- **Token Limit Management (AF-1B):** when a document exceeds the model's context window, the system breaks down the document into smaller pieces of text, processes each piece, and then synthesises the processed pieces back together.

- **Post-Conditions:**

- Summaries reference key document sections where applicable.
- On average, the system takes less than 30 seconds to process a document.

3.1.2 UC-02: Document Classification

- **Actors:** Administrative assistant, Records manager.

- **Goal:** Automatically categorise documents based on AI-inferred or user-defined schemas.

- **Preconditions:**

- User has provided documents and indicated the task they want the system to perform.

- **Main Flow**

1. The user uploads a diversified batch of documents.
2. The Orchestration Agent identifies the classification intent, and directs the task to the Classification Agent.
3. The specialised Classification Agent analyses the document's content, and determines the document's optimal taxonomic classification.
4. The system displays the categorised documents to the user.

- **Alternative Flows**

- **Semantic Rejection (AF-2A):** if a document does not align with any valid category, the agent rejects the classification, provides a justification (e.g., 'Incompatible Document Content'), and indicates a possible solution.

- **Post-Conditions**

- Categorised results are archived in the chat history.
- For common document types, the system has a documented accuracy of >85% when classifying documents.

3.1.3 UC-03: Data Extraction

- **Actors:** Accountant, Tax specialist.

- **Goal:** Transform unstructured text into validated, structured data schemas.

- **Preconditions:**

- The documents contain necessary fields (e.g., invoice number, date, total).
- The user optionally specifies the fields they wish to have extracted.

- **Main Flow:**

1. The user uploads documents and a list of target fields (e.g., 'Total Amount').
2. The Orchestration Agent assigns the user's request to the Data Extraction Agent.
3. The Data Extraction Agent searches through the documents for the required fields, and interprets them.
4. Extracted data is presented in a structured, standardised format.

- **Alternative Flows:**
 - **Incomplete Field Recovery (AF-3A):** if a primary field is missing, the agent notifies the user while successfully extracting all secondary available metadata.
 - **Heuristic Validation (AF-3B):** if extraction returns non-compliant data types (e.g., alphabetic characters in a currency field), the agent re-tries the prompt to resolve the ambiguity.
- **Post-Conditions:**
 - The user has access to the extracted data in the chat interface, after validation against basic type-checks.
 - Extraction accuracy exceeds 90% for clear digital sources.

3.2 System Requirements

This section describes the requirements necessary to fulfil the objectives of the AWOS. These are categorised into Functional Requirements, which specify the technical capabilities of the system; Non-Functional Requirements, which define quality and performance constraints; and System Constraints, which outline the architectural boundaries and technology stack choices.

3.2.1 Functional Requirements (FR)

Functional requirements define the specific capabilities the system provides to satisfy the objectives identified in the Use Cases.

ID	Requirement Description	Use Cases
FR-01	The system must utilise an Orchestration Agent to perform semantic intent analysis on natural language queries.	UC-01, UC-02, UC-03
FR-02	The system must route to specialised agents (Summarisation, Classification, or Extraction) based on the identified intent.	UC-01, UC-02, UC-03
FR-03	The system must support the ingestion and text extraction of pdf , txt , docx , csv , xls , and xlsx files.	UC-01, UC-02, UC-03
FR-04	The system must generate concise document summaries that capture primary insights and key entities.	UC-01

FR-05	The system must handle large documents by segmenting text into chunks that fit within LLM context windows (recursive synthesis).	UC-01
FR-06	The system must classify documents into user-defined or AI-inferred categories with high confidence precision.	UC-02
FR-07	The system must reject classification tasks that do not align with available document content and provide justification.	UC-02
FR-08	The system must extract user-specified data fields and transform them into validated, structured schemas.	UC-03
FR-09	The system must perform heuristic validation on extracted data to ensure data type integrity.	UC-03
FR-10	The system must provide a chat-based interface for interaction, file uploading, and real-time result display.	UC-01, UC-02, UC-03
FR-11	The system must support batch processing, allowing multiple files to be processed individually within a single request.	UC-01, UC-02, UC-03
FR-12	The system must support multi-turn conversational interactions, enabling the user to upload files and execute different specialised tasks sequentially within the same chat thread.	UC-01, UC-02, UC-03

Table 3.1: Functional Requirements

3.2.2 Non-Functional Requirements (NFR)

Performance and Accuracy

ID	Requirement Description
NFR-01	Responsiveness: Average processing time for a standard document summary, classification or data extraction must be < 30 seconds.
NFR-02	Classification Accuracy: The system must achieve > 85% accuracy in matching documents to correct categories.
NFR-03	Extraction Precision: The system must correctly identify and extract target fields in > 90% of cases for digital sources.
NFR-04	Throughput: The system must be capable of handling batches of up to 100 (1-2 pages) documents without significant latency degradation.

Table 3.2: Performance and Accuracy Requirements

Reliability and Usability

ID	Requirement Description
NFR-05	Fault Tolerance: Failure in processing one document in a batch must not terminate the entire workflow.
NFR-06	Intent Robustness: The Orchestration Agent must correctly route at least 95% of clear natural language prompts to the correct specialised agent.
NFR-07	Availability: The system must remain accessible for the duration of an active user session, with no planned downtime during operation.

Table 3.3: Reliability and Usability Requirements

Security and Data Privacy

ID	Requirement Description
NFR-08	Encryption: All communication between the orchestrator and the LLM APIs must be encrypted to ensure data privacy.
NFR-09	Statelessness: To mitigate the risk of information leakage, the system must process all documents in volatile memory. User-uploaded files must never be persisted to non-volatile storage and all temporary buffers must be cleared upon completion of the workflow.

Table 3.4: Security Requirements

Maintainability and Scalability

ID	Requirement Description
NFR-10	Maintainability: The system architecture must be decoupled, allowing individual specialised agents (e.g., Summarisation) to be updated or replaced without affecting the Orchestration layer.
NFR-11	Scalability: The system must handle an increase in document complexity (up to 25MB per file) without a failure rate exceeding 5%.

Table 3.5: Maintainability and Scalability Requirements

Compliance and Auditability

ID	Requirement Description
NFR-12	Auditability: The system must keep a transparent execution log for every request processed to allow for manual verification of AI-driven decisions.
NFR-13	Format Compliance: Output must be presented in structured, human-readable formats.

Table 3.6: Compliance Requirements

3.2.3 System Constraints (SC)

ID	Requirement Description
SC-01	Directed Agentic Orchestration: The system must utilise an agentic orchestration pattern constrained within a directed workflow. This architecture ensures intelligent decision making and LLM-driven task routing.
SC-02	LLM-Inference Dependency: The core reasoning for intent analysis and task execution must be delegated to specialised external LLM providers to provide the high-speed inference required for real-time interaction.
SC-03	Input Modality Limitations: The system is strictly constrained to digitally generated, text-based file formats. Scanned documents, handwritten notes, or image-heavy PDFs requiring Optical Character Recognition (OCR) are explicitly excluded from the current architectural scope.
SC-04	Data Privacy Constraint: The system must handle sensitive document fields (e.g., account numbers, names) required for different tasks. As processing involves external cloud-based APIs, the system is constrained by the data privacy and residency policies of third-party providers, establishing a mandatory privacy constraint for the project's data flow.
SC-05	Deterministic State Management: Session consistency must be achieved through the workflow's in-memory context handling and direct API calls. This design ensures reproducible, traceable workflows with verifiable execution paths for audit and debugging purposes.

Table 3.7: System Constraints

3.3 System Specifications

This section specifies the technical contracts, data formats, and operational parameters that govern system components. It establishes technology-agnostic behavioural requirements that enable component interoperability, fault isolation, and independent testing.

3.3.1 Component and Layer Specifications

The system utilises a three-tier modular architecture designed for strict fault isolation (NFR-05) and decoupled maintainability (NFR-10).

Component	Technical Specification
Interface Layer	Must provide a chat-based entry point for synchronous file ingestion and asynchronous result rendering (FR-10).
Orchestrator	Logic: Single-pass semantic intent analysis via external LLM API. Must map user intent to specialised agent workflows with $\geq 95\%$ accuracy (FR-01, NFR-06).
Summarisation Agent	Logic: Recursive synthesis for files up to 25MB (NFR-11). Must output thematic, structured summaries (FR-04, FR-05).
Classification Agent	Logic: Semantic content matching against user-defined taxonomies. Must provide rejection justification if documents fall outside defined schemas (FR-06, FR-07).
Extraction Agent	Logic: Field localization and type-aware value extraction. Must apply heuristic validation (FR-09) to ensure 90% precision (NFR-03).

Table 3.8: System Component and Layer Specifications

3.3.2 Data Flow Specification

The system processes requests through a deterministic five-stage pipeline:

1. **Ingestion:** User uploads files via chat interface. System validates format compliance (FR-03: pdf, txt, docx, csv, xls, xlsx).
2. **Text Extraction:** Binary content converted to plain text. Documents exceeding LLM context limits undergo recursive chunking (FR-05).
3. **Intent Analysis:** Natural language prompt analysed to determine task type with $\geq 95\%$ routing accuracy (NFR-06).
4. **Task Execution:** Appropriate specialised agent processes extracted text. Agent must refuse out-of-scope tasks (AF-2A).
5. **Result Rendering:** Output formatted per NFR-13 and displayed in chat interface (FR-10).

Each stage must complete successfully or trigger fault recovery (NFR-05).

3.3.3 Requirement Traceability Matrix

This matrix provides a mapping between the system specification and the requirements established in the previous section, ensuring full architectural coverage.

Specification Area	Requirement ID	Verification Method
Workflow Coordination	FR-11, FR-12, SC-05	Multi-turn session consistency and batch loop verification.
Intent Analysis	FR-01, FR-02, NFR-06	Intent routing accuracy testing via varied natural language prompts.
Scalability	FR-03, FR-05, NFR-11	Stress testing with 25MB files across different supported formats.
Privacy & Security	NFR-08, NFR-09, SC-04	Audit of volatile memory usage and API encryption.
Performance	NFR-01, NFR-04, SC-02	Latency measurement (< 30s average per document) for 100-document batch loads.

Table 3.9: Requirement Traceability and Verification Matrix

3.3.4 Workflow State and Error Handling

To ensure deterministic execution (SC-05), the orchestration workflow maintains a strictly defined state machine and error-recovery protocol.

Workflow State Variables

State Variable	Specification / Lifecycle
Session ID	Unique ID for multi-turn threads; persists for the duration of the chat (FR-12).
Context Buffer	Volatile storage for document text; purged immediately upon task completion (NFR-09).
intent_enum	One of <code>summarisation</code> , <code>classification</code> , <code>extraction</code> ; determined by Orchestrator.
retry_counter	Integer (Max: 1); tracks Heuristic Validation attempts for extraction (AF-3B).

Table 3.10: Workflow State Specifications

Fault Tolerance and Recovery (NFR-05)

- **Batch Fault Isolation:** If an individual document fails processing (e.g., corruption), the system must isolate the error and continue the execution.
- **Heuristic Retry Logic:** Upon detection of a type-mismatch (e.g., text in a currency field), the agent must perform a recursive retry with an adjusted prompt (Max 1 attempt).
- **Graceful Degradation:** If an agent task is refused, the system must return a status message with specific exclusion reasons to the user for clarification.

3.3.5 Audit and Monitoring Specification

To satisfy the auditability requirements (NFR-12), the system must log technical metadata for every execution cycle.

Log Parameter	Specification
Routing Log	Records the user prompt against the selected agent to monitor accuracy (NFR-06).
Latency Stamp	Captures execution duration per agent call to verify the < 30s target (NFR-01).
Audit Trail	Complete workflow execution records including routing decisions and API payloads, stored within the orchestration platform for post-execution analysis.

Table 3.11: Audit and Monitoring Specifications

3.4 System Design and Architecture

3.4.1 Design Philosophy: Deterministic Orchestration

The core philosophy of this system is **Directed Agentic Orchestration**. Unlike fully autonomous agent frameworks such as Planner-Executor-Evaluator-Judge (PEEJ) workflow, which can experience unbounded execution through iterative re-planning, this design uses explicit routing with bounded retry attempts (AF-3B).

The goal is to provide the intelligence of an LLM for semantic intent analysis while maintaining the determinism of traditional software engineering. By constraining agent transitions within a predefined workflow, the system ensures comprehensive traceability with complete execution logs (NFR-12) and predictable execution paths (SC-01, SC-05).

Architectural Trade-off Analysis:

Initial architectural exploration considered the PEEJ framework for its sophisticated multi-step reasoning capabilities. Preliminary analysis revealed:

- **Cost implications:** 4-8 LLM API calls per document vs 1-3 in directed routing (representing an approximate 400% increase in operational expenditure).
- **Debugging complexity:** Non-deterministic re-planning loops difficult to trace and reproduce.
- **State management overhead:** Requires coordination between PEEJ components.

The directed orchestrator pattern was selected because it achieves the core objective of intelligent task routing, while maintaining high production determinism and cost efficiency.

3.4.2 Architectural Design Decisions

Workflow Orchestration Approach

The system requires a workflow orchestration mechanism that satisfies three design criteria derived from the system requirements:

- **Auditability (NFR-12):** Native execution history and logging to support post-execution analysis of routing decisions and agent behaviour.
- **Statelessness (NFR-09):** In-memory processing of user documents without persistence to non-volatile storage.
- **Decoupled Maintainability (NFR-10):** Visual debugging and modular workflow structure to allow independent updates to specialised agents without affecting orchestration logic.

A node-based workflow automation platform was chosen over custom framework development because it provides these capabilities natively, reducing development overhead and supporting reproducible workflow execution (SC-05). The specific platform selected and the rationale for its choice over alternative approaches are discussed in the Implementation chapter.

Inference Strategy: The Multi-API Hybrid Model

The architecture employs a task-optimised inference strategy where different specialised agents delegate reasoning to different LLM providers based on task characteristics. This hybrid approach is driven by two competing design requirements that cannot be satisfied by a single provider:

- **Low-latency inference** for the orchestrator and data extraction agent, which must complete their tasks within the $< 30s$ responsiveness target (NFR-01) to support real-time user interaction.
- **Extended context capacity** for the summarisation agent, which must process large documents up to 25MB (NFR-11) without losing semantic coherence.

The separation of concerns across multiple inference providers allows each agent to delegate to the most appropriate backend for its workload, balancing cost, latency, and context window capability. Specific provider and model selections, along with empirical justification, are detailed in the Implementation and Evaluation chapters.

3.4.3 System-Specific Design Patterns

Multi-Level Task Refusal Pattern

The system implements task refusal at two levels to ensure quality and maintain functional boundary adherence:

Level 1 - Orchestrator Task Availability Check:

The Orchestrator Agent rejects requests for unimplemented specialised agents.

Example: User asks to “compare these two contracts”

- Orchestrator recognises comparison is not an available task type
- Returns: “Comparison is not currently supported. Available tasks: Summarisation, Classification, Extraction”
- User redirected to available functionality
- Prevents system from attempting unsupported operations

Level 2 - Specialised Agent Semantic Rejection (AF-2A):

Each specialised agent validates document compatibility with the requested task.

Example: User uploads restaurant menu, requests classification as “electricity bill or gas bill”

- Orchestrator correctly routes to Classification Agent
- Classification Agent analyses document content
- Agent determines document type incompatible with requested categories
- Returns: “Document does not match requested categories. This appears to be a restaurant menu, not a utility bill.”
- The user receives semantic rejection with clear justification

Benefits:

- **Quality assurance:** Prevents low-quality outputs from mismatched document-task combinations
- **Clear error messaging:** Users understand why their request cannot be fulfilled
- **Scope boundaries:** System cannot be coerced into performing tasks outside its designed capabilities

Heuristic Validation with Automatic Retry (AF-3B)

The Data Extraction Agent performs type validation on extracted fields:

- Currency fields must be numeric
- Dates must match expected format patterns
- If validation fails, agent retries with refined prompt (max 1 attempt)
- Example: “Extract only the numeric value for Total Amount”

This improves data type compliance and reduces the need for manual correction of extraction outputs (UC-03).

3.4.4 System Architecture

This subsection presents the system architecture through two complementary views: a component diagram illustrating the structural separation of concerns, and a sequence diagram showing the temporal flow of a request.

Component Architecture

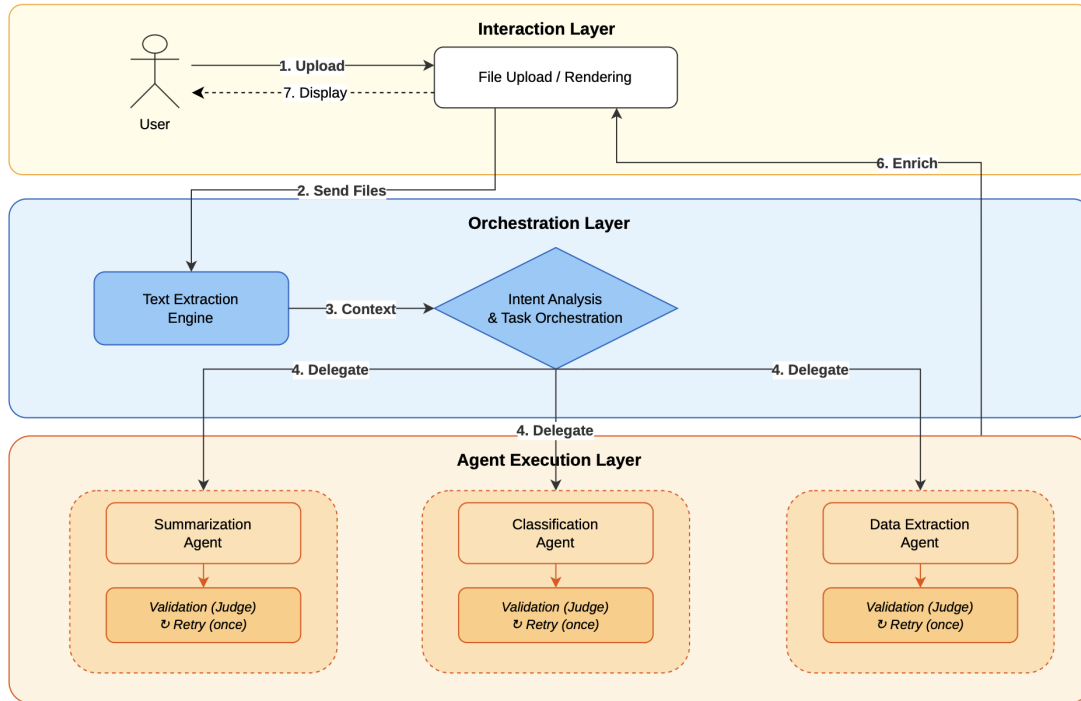


Figure 3.1: Three-tier component architecture of AWOS, showing the Interaction, Orchestration, and Agent Execution layers with numbered data flow steps.

The architecture enforces strict separation of concerns:

- **Interface Layer:** Handles user interaction and file upload (FR-10)
- **Orchestration Layer:** Routes requests based on intent (FR-01, FR-02)
- **Agent Execution Layer:** Performs specialised tasks (FR-04, FR-06, FR-08)

Execution Flow

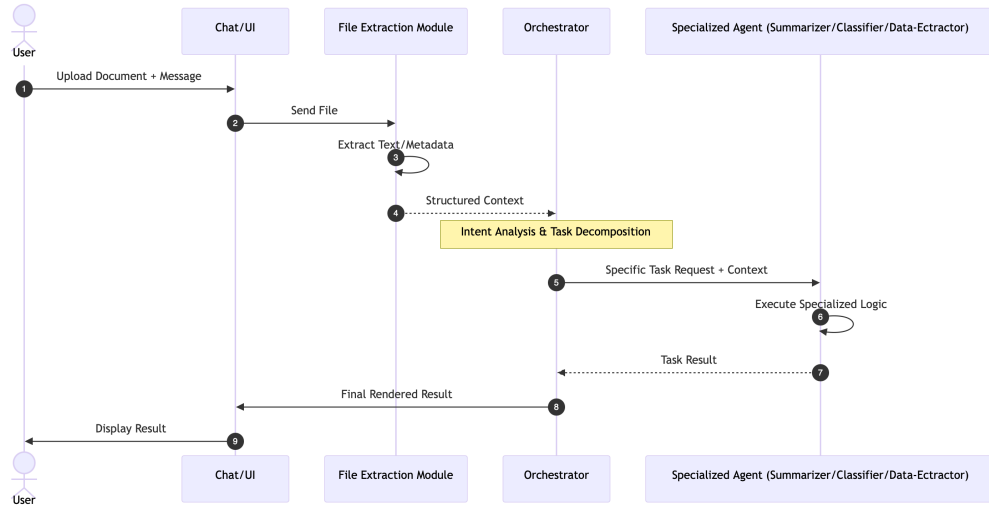


Figure 3.2: Sequence diagram illustrating the end-to-end execution flow from document upload through text extraction, intent analysis, specialised agent execution, and result rendering.

This deterministic flow ensures reproducible execution (SC-05) while maintaining the flexibility to add new agents without modifying orchestration logic (NFR-10).

Chapter 4

Implementation

This chapter documents the implementation of the Autonomous Workflow Orchestration System (AWOS) as realised from the design specifications in Chapter 3. Covers development methodology, technology selection, component implementation, engineering challenges, and functional verification, with a final focus on deployment and reproducibility.

4.1 Development Methodology

AWOS has been developed in accordance with a phased iterative process. The use of a waterfall development process was dismissed because LLMs are inherently stochastic; therefore it is necessary to implement empirical cycles of *prompt engineering* $\rightarrow$ *failure analysis* $\rightarrow$ *refinement* to establish reliability for interaction between multiple agents as part of a multi-agent workflow.

4.1.1 Development Process by Phase

The implementation of AWOS has been divided into five distinct logical phases (see table 4.1). These phases allow each level of the hierarchical structure (i.e., from infrastructure through to intent orchestrations) to be tested and validated prior to integrating dependent agents.

Phase	Focus	Product/Deliverables
1	Environment & Infrastructure	Deploy containerised orchestration environment and relational database. Configure volatile binary memory buffers.
2	Orchestration Layer	Implement intent-analysis logic and level 1 task-refuse mechanism.
3	Specialised Agents	Develop task-specific modules for summarisation, classification and data extraction using hybrid inference frameworks.
4	Integration & Interface	Construction of the workflow-integrated chat interface and multi-turn session management.
5	Optimisation & Validation	Perform systematic prompt tuning against the Sample Dataset to align model outputs with NFR target requirements.

Table 4.1: Phased Development Approach

Each phase concluded with an explicit validation gate before the subsequent phase commenced, ensuring that no defect from an invalidated layer propagated as a cascade failure into dependent components.

4.1.2 Version Control and Engineering Standards

In order to provide a high degree of professionalism in a low-code development environment, all standard DevOps practices were followed on the orchestration layer:

- **Configuration as Code:** All workflow logic was treated as software code and exported as versioned JSON assets. Therefore, it could be tracked via Git and if necessary rolled back at any time throughout the developmental cycle.
- **Prompt Management:** Instructions were isolated to their own configuration nodes to isolate prompt logic from the flow of execution. This minimised prompt drift and ensured that instruction updates remained auditable.
- **Environmental Parity:** Automated deployment scripts were utilised to pin specific versions of both the orchestration engine and database images. Thus, the reproduction and portability of results can be assured regardless of differences in host environments.

4.2 Technology Stack

This section explains why particular technology choices were made for the implementation, to fulfill the requirements developed during the design stage.

4.2.1 Workflow Orchestration: n8n

n8n was selected as the workflow orchestration engine for its built-in execution auditability of executions (NFR-12) and also handles JSON natively [49].

- **Technical Justification:** n8n enables real-time, visual debugging of the payload that are being passed between agents. This enabled faster iteration on the output of LLM’s than would have been possible using traditional programmatic logging.
- **Privacy by Design:** The system uses n8n’s internal memory buffer to process document content so that no user data is persisted to non-volatile storage (NFR-09).

Alternatives	Drawbacks	Result
FastAPI/Python	Excessive boilerplate code needed to implement retry logic for asynchronous APIs and maintain state.	Rejected: Excessive development overhead for state management and execution history tracking.
LangChain/CrewAI	Opaque execution model limits visual auditability and makes agent-to-agent state changes difficult to inspect.	Rejected: Too opaque to debug state changes at agent-to-agent level.

Table 4.2: Platform alternatives for Orchestration

4.2.2 Inference Strategy: Hybrid API Architecture

To find an optimal balance between the capabilities for reasoning, the size of the context window and costs; a hybrid strategy was used to select the provider. Each platform’s unique features were taken into consideration when selecting the platforms, along with the availability of free tiers. Cost-efficiency was a primary selection criterion, both to ensure the project’s development feasibility and to demonstrate that the architecture can be deployed without prohibitive operational expenditure, a deliberate contrast to enterprise-scale systems relying on premium proprietary APIs.

Two complementary providers were selected for the hybrid strategy:

- **Google AI Studio (Gemini Models):** Offers free-tier access to Google’s Gemini Models Family. While the rate limits on the free-tier are variable by model (e.g., Gemini 2.5 Flash Lite provides up to 20 requests per day while more powerful models like Gemini 2.5 Pro need to be purchased). There are token-per-minute limits on the free tier which allow for approximately 250K tokens/minute depending on the model. Although in theory the context window of the underlying Gemini Models can span 2M+ tokens, in practice, free-

tier limitations constrain the amount of data that can be processed per request within a specific time frame [17].

- **Groq LPU Inference:** Houses various open source models (Llama 3.1/3.3, Qwen 3, GPT-OSS versions) on Groq’s custom-built Language Processing Unit (LPU). The Groq’s free tier provides support for 30-60 requests per minute and 6-30K tokens per minute, depending on the model. Due to its high inference throughput compared to GPU-based implementations, Groq is well-suited to latency-sensitive workflows where fast response time is prioritised over large single-request context processing [18].

These two providers are complementary rather than redundant: Google AI Studio offers access to models with very large context windows suited to processing lengthy documents, while Groq offers faster inference suited to quick turnaround tasks. The combination allows the architecture to allocate workloads based on whichever constraint dominates each task.

Comparison of Alternatives

The selection process explicitly evaluated industry-leading proprietary models, which were ultimately rejected based on cost-efficiency and accessibility constraints.

Alternative	Limitations	Outcome
OpenAI (GPT-Models) & Anthropic (Claude-Models)	No persistent free API tier, making costs prohibitive for iterative testing across large document volumes.	Rejected: Prohibitive cost for iterative, high-volume document testing, as per project’s constraints.
Local LLM (Ollama)	High local hardware dependency (GPU, CPU & RAM); insufficient precision for structured data extraction on consumer-grade hardware without dedicated GPU resources.	Rejected: Insufficient precision observed during preliminary testing on consumer-grade hardware; fails to satisfy NFR-02 and NFR-03 targets without dedicated GPU resources.

Table 4.3: Inference Strategy Alternatives and Cost-Based Selection

4.2.3 Infrastructure and Data Persistence: Docker and PostgreSQL

The deployment and persistence layers were selected to ensure system portability and auditability.

- **Docker and Docker Compose:** The use of containerisation allowed for the bundling of the orchestration engine and database into one deployable unit. To provide an easy way to describe how the containers interact with each other and the application, **Docker Compose** was used [23].

- **PostgreSQL:** A **PostgreSQL** database was also added to allow n8n to store its execution history along with chat metadata. As a result, it will satisfy the NFR-12 of having a persistent audit trail of which agent executed what task. However, user-uploaded document contents remain exclusively in volatile memory and are cleared on workflow completion or container restart, ensuring NFR-09 compliance [43].

4.3 System Implementation

This section describes the implementation of each architectural component defined in Chapter 3, focusing on the non-trivial decisions made during development. The entire system is implemented as a single unified n8n workflow, with logically separated regions corresponding to the three-tier architecture.

4.3.1 Workflow Structure

The system has been developed as an n8n Workflow made up of six logically distinguishable areas, each visibly separated on the n8n canvas (see Fig. 4.1):

1. **User Chat Interface:** User entry point and source of file upload detection.
2. **File Extraction Engine:** Performs format-specific text extraction on the uploaded documents.
3. **Orchestrator Agent:** Analyses the intent of the document and sends the requests to the relevant special agent using a structured output parser.
4. **Summariser Agent:** Generates per-document summaries within token constraints.
5. **Classifier Agent:** Classifies documents and infers categories.
6. **Data Extractor Agent:** Performs structured field extraction from documents.

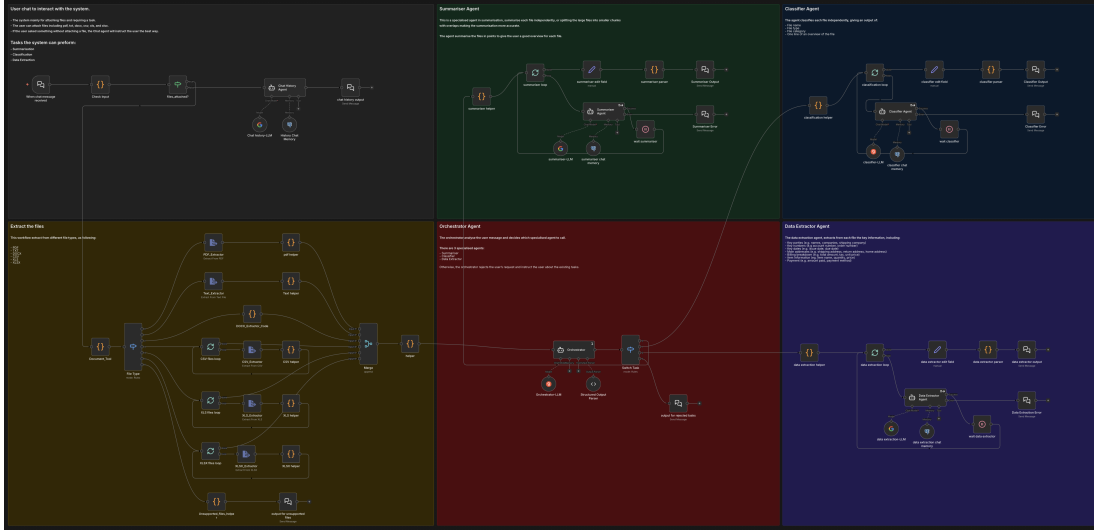


Figure 4.1: Overview of n8n workflow that implements a 3-tiered architecture. Each colour block represents a logically distinct area.

The workflow executes sequentially: incoming messages are first evaluated for file attachments, which are routed to the File Extraction Engine if present. Extracted text is forwarded to the Orchestrator Agent, which routes the request to the appropriate specialised agent via a Switch node. If no files are attached, the message is passed directly to the Chat History Agent for conversational handling.

4.3.2 Text Extraction Engine

The **Text Extraction Engine** extracts the uploaded binary files into plain text compatible with LLM-API processing.

Multi-Format Ingestion

Format	Extraction Method	Implementation Details
PDF	n8n Extract From PDF node	Native binary → text extraction.
TXT	n8n Extract From TXT node	Plain text pass-through; no conversion required.
DOCX	JavaScript Code node (Mammoth library)	A custom implementation was required since n8n does not contain a native DOCX extraction node. Mammoth converts DOCX to plain text, maintaining the document structure.
CSV/XLS/XLSX	n8n Extract from Spreadsheet file node	Three separate nodes process each format into structured row and column format.

Table 4.4: Document Formats Used in Extraction Module

All extraction paths feed into format-specific helper nodes that normalise extracted text into a consistent format prior to merging them at the Merge node. The Merge node aggregates all extracted content into a unified payload regardless of the original file format, so downstream agents receive a unified text input.

Unsupported file formats are processed by a dedicated fallback path that provides users with an informative error message via the chat interface, instead of silently failing.

4.3.3 Orchestration Agent

The Orchestration Agent is the central routing component of the system. It receives the user's natural language request and determines which specialised agent should handle the task.

Intent Classification

The orchestrator uses an LLM to perform semantic intent analysis on the user's message. The prompt is designed to constrain the model's output to one of four predefined categories:

```
User message: {{ ('When chat message received ').first().json.chatInput }}
Analyse the user message and decide which task from the following is required.
- Summarisation
- Classification
- Data Extraction
- Otherwise
```

Listing 4.1: Orchestrator Intent Classification Prompt

The LLM's response is passed through a Structured Output Parser tool. This forces the orchestrator to generate machine-readable routing decision instead of free-form text, and reduces the risk of routing failure due to ambiguous LLM output.

Task Routing

A **Switch** node receives the parsed intent and routes the workflow execution to the corresponding specialised agent. The Switch node evaluates the orchestrator's output against three defined paths (Summarisation, Classification, Data Extraction). If the intent does not match any defined task, the request is directed to a rejection output path that informs the user of the available capabilities.

This rejection mechanism prevents the system from attempting unsupported operations by intercepting unrecognised intents before they reach any specialised agent.

4.3.4 Specialised Agents

Each specialised agent follows a standard architecture pattern: A process loop which goes over all documents uploaded; An LLM-node where it performs the required actions; A memory-chat node (for a session) where all context information from chats are stored; And helper-nodes for output-formatting. Additionally, the system is capable to process many files at once (batch-processing), because there is a additional helper-node in charge of managing the limitations imposed on the token limit by the underlying LLM. Based on configuration options, this helper either divides large files into overlapping segments or merges multiple small files into a single request. The Specialised Agents work independently and do not communicate with each other directly, thus supporting centralised orchestration as discussed in literature review.

All three specialised agents currently utilise Google AI Studio (Gemini) and Groq LPU (GPT-OSS & LLAMA 4) models for inference. The specific model-to-agent allocation is revisited in the Evaluation chapter, where alternative configurations are empirically compared.

Summarisation Agent

The Summarisation Agent processes uploaded files within a loop structure, using batches based on the LLM’s limitations for requests and tokens. For each batch of documents, the agent invokes the LLM with a summarisation-specific prompt and generates a structured summary. The agent uses a dedicated Summariser Chat Memory node to maintain conversational context across multi-turn interactions.

You are a summarisation agent. Summarise each document provided in this batch independently and in depth.

- Be thorough; the summary should give a useful understanding without reading the original
- Never fabricate content not present in the document
- Separate each file’s summary with a markdown divider

Listing 4.2: Summarisation Agent Prompt (simplified)

The Summarisation Agent is instructed to produce hierarchical markdown outputs. Specifically: beginning with **Document Essence** sentence followed by bullet points such as key findings. Ensuring that even large documents such as a 25-page scientific paper, providing users with digestible overviews that preserve the logical hierarchy of the original document.

Classification Agent

The Classification Agent categorises each document using either user-defined labels or AI-inferred taxonomies. Target categories are passed from the Orchestrator to the agent as a JSON variable; if no categories are specified in the user’s initial request, the agent is instructed to perform zero-shot inference to determine the most logical classification. To ensure system consistency, the agent’s output is strictly constrained to the following JSON schema:

To optimise token consumption and minimise latency (NFR-01) during the classification phase, the specialised agent employs a **context sampling strategy**. When classifying spreadsheets, it will only examine the first 5 rows of data since this provides enough structural context to classify the document by domain, and provide sufficient contextual information for classification. On the other hand, when classifying documents based on their content, it performs the task on the first 25% of the total file content. The assumption is that most of the relevant metadata and subject matter found in professional documents reside in the upper sections of the documents. Processing only the initial portion of each document reduces token consumption while preserving classification accuracy, since domain-relevant metadata typically appears in the early sections of professional documents.

```
{
  "filename": "{ json.filename }",
  "document_type": "<type>",
  "category": "<subject or domain>",
  "info": "<one sentence describing what this document is about>"
}
```

Listing 4.3: Classification Agent JSON Output Schema

This structured response allows n8n to route the classification results directly to the chat interface while maintaining a standardised format in the PostgreSQL execution log. By requiring a one-sentence “info” field, the agent also provides a brief semantic context for the classification decision, aiding user verifiability.

Data Extraction Agent

The Data Extraction Agent performs structured retrieval of specific data points from the ingested text. The agent’s prompt is constructed to identify primary information clusters, such as reference numbers (e.g., account number, id number), financial data (e.g., invoice totals, tax

amounts), and payment metadata (e.g., payment methods or codes).

```
{
  "filename": "{{ json.filename }}",
  "document_type": "<Invoice | Bill | Receipt | define type if unclear>",
  "extracted_data": {
    "dates": {},
    "reference_numbers": {},
    "parties": {},
    "financials": {},
    "line_items": [],
    "payment_details": {}
  }
}
```

Listing 4.4: Data Extraction Agent JSON Output Schema

The extraction workflow utilises a dedicated JSON parser node to convert the LLM’s response into a flat key-value pair. This deterministically formatted output guarantees that extracted fields may be immediately rendered visible to user or relayed downstream to spreadsheet nodes for reporting purposes.

4.3.5 Heuristic Validation Implementation Review

Although the heuristic validation and automatic retry mechanism (AF-3B), which re-attempts a task when agent output fails schema compliance checks, was implemented and is operational, empirical observation during testing revealed a fundamental limitation in its practical utility. The retry mechanism provides meaningful benefit only when the initial failure stems from prompt ambiguity that a refined instruction can resolve. In practice, two distinct behaviours were observed. Models that consistently produced well-structured outputs rarely triggered the validation check, rendering the retry logic redundant for high-performing configurations. Conversely, models that produced non-compliant output on the first attempt were found to repeat the same error on retry, suggesting that the failure originates from a model capability constraint rather than prompt ambiguity. As a result, the mechanism functions effectively as a safety net against isolated formatting errors but does not provide systematic improvement in overall output reliability across agents.

4.3.6 Exploration of Alternative Approach: MCP Integration

As discussed in Section 2.4, the Model Context Protocol (MCP) has been examined as a means of coordinating agent communication. In order to confirm the theoretical concerns documented in the literature review with data-driven evidence, an alternate implementation utilizing n8n’s MCP tool integration was implemented.

The MCP-based method had a significantly greater failure rate than the methodology utilizing direct orchestration. There were three major problems with this method. First, the protocol added an extra API call to each task resulting in nearly double the number of tokens being consumed relative to the direct routing method. Second, the LLM frequently misdirected tasks or denied valid requests when it was working under the MCP abstraction layer. Third, the protocol did not function well with large payloads of documents.

In conclusion, these empirical findings supported the theoretical evaluation conducted in Section 2.4 confirming the architectural design decision to utilise direct n8n workflow orchestration over MCP-based coordination. Direct routing uses fewer tokens, is more predictable, and yields better routing performance.

4.3.7 Chat Interface and Session Management

The user-facing interface is provided by n8n’s built-in Chat Trigger node, which serves as the system’s single entry point. The Chat Trigger handles both text messages and file uploads natively, eliminating the need for a custom frontend application.

Interaction Flow

When a message is received, the workflow first evaluates whether files are attached using a conditional check node. This branching logic determines the execution path:

- **Files attached:** The workflow continues to the Text Extraction Engine after which it continues with orchestration and agent execution.
- **No files attached:** The message is passed to a Chat History Agent, which uses a designated LLM and History Chat Memory to create conversational responses that are based upon all prior interactions within that user’s current session.

Session Management

Each specialised agent has its own Chat Memory node allowing for multi-turn conversation inside a single session (FR-12). Therefore, when users upload files and execute a task they can query agents regarding those files’ output without having to re-upload them again.

User-uploaded document content is retained in volatile memory while being processed but not persisted to the PostgreSQL database. Metadata related to execution history (such as routing decisions, timestamps, agent selections) is stored in the database for auditing purposes (NFR-12), therefore complying with NFR-09 and retaining user-uploaded document content solely in volatile memory.

4.4 Technical Challenges and Implementation Strategies

This section analyses the non-trivial engineering obstacles encountered during the implementation of AWOS and the strategies developed to resolve them.

4.4.1 Challenge 1: Semantic Loss in Long-Context Summarisation

- **Problem:** While Gemini models allow up to approximately 250K tokens per request under the Google AI Studio free tier, other providers such as Groq’s free tier restrict Llama-based models to roughly 8K tokens per request. Large documents therefore cannot be processed in a single pass, and the free-tier rate limits further demand a strategy to maintain API stability during batch processing.
- **Strategy:** The system implements a **Recursive Synthesis** approach with an overlap consideration. Large files are split into chunks with a 5% overlap, which are then processed via n8n loop nodes to generate intermediate summaries, which are then aggregated. This balances global understanding with granular detail preservation. To further manage free-tier rate constraints during batch processing, a wait node is introduced within specialised agents loop. Although this marginally increases per-document latency, it prevents request failures caused by exceeding the provider’s token-per-minute ceiling during multi-document sessions.

4.4.2 Challenge 2: Achieving Determinism in a Stochastic System

- **Problem:** NFR-06 requires the Orchestration Agent to route requests with at least 95% accuracy. LLMs are inherently stochastic; given the same input, the model may pro-

duce syntactically distinct responses that carry identical semantic intent. For example, a summarisation request could yield “*Summary*”, “*I will summarise your request*”, or “*Summarisation is the correct action*”; all equivalent in meaning, but each likely to produce a different outcome under exact string matching at the Switch node.

- **Strategy:** A two-layer constraint mechanism was implemented to bound the model’s output space. First, the LLM was adjusted to a consistent response setting that eliminates variability between repeated requests and guarantees that the most likely answer will be returned every time for a specific input. The second layer includes passing the output of the model into a Structured Output Parser node that forces adherence to a predetermined schema and rejects any responses that do not adhere to one of the predefined intent labels. By combining these constraints, the probabilistic output of the model has been constrained to fit into a deterministic decision making space, allowing it to work well with the downstream routing logic while maintaining semantic flexibility with regard to the input side of the LLM.

4.4.3 Challenge 3: Balancing Auditability with Statelessness

- **Problem:** A conflict exists between NFR-12 (Auditability) and NFR-09 (Statelessness). Standard logging of n8n execution data would inadvertently store document content in the database.
- **Strategy:** The system adopts a **Metadata-Content Decoupling** architecture. The PostgreSQL database is configured to store Execution Metadata (timestamps, routing decisions, and agent IDs). While Document Content is stored in volatile memory, therefore, the system can always provide an audit trail, and reduces risk of this audit trail exposing data to anyone for extended periods.

4.5 Testing and Validation

Testing ensures the functional integrity of the orchestration logic and the stability of inter-agent communication paths. Unlike traditional software code-bases, n8n does not expose a native unit-testing framework; individual nodes are not directly callable in isolation, and there is no built-in assertion or mocking API. Consequently, a custom two-component verification strategy was developed to confirm that each module operates correctly before the system was

subjected to qualitative evaluation. Importantly, this chapter concerns only functional correctness, whether the system behaves as specified, and not the agent’s output quality, which is addressed in Chapter 5.

4.5.1 Automated Functional Testing via Manual Triggers

n8n provides a **Manual Trigger** node that executes a workflow on demand with a static, pre-defined data payload, bypassing the live Chat Trigger. This mechanism was used as a controlled substitute for unit testing, allowing individual modules to be tested with known inputs and their outputs inspected before downstream integration.

The following modules were verified in this manner:

- **File Extraction Engine:** Each supported format (PDF, TXT, DOCX, CSV, XLS/XLSX) was tested by injecting a static payload consisting of all supported and unsupported file types (file names ending with extension) into the extraction pipeline. This ensures that the system goes through each supported flow correctly (see Figure 4.2).
- **Orchestration Routing Logic:** Four static test prompts were constructed, each of these four is targeted one of the three specialised agents (Summarisation, Classification, Data Extraction), and one representing an out-of-scope request. These tests requests were entered to the testing workflow using the Manual Trigger node. The output of the Switch node was checked to ensure that all Test Requests had been forwarded to the appropriate path.

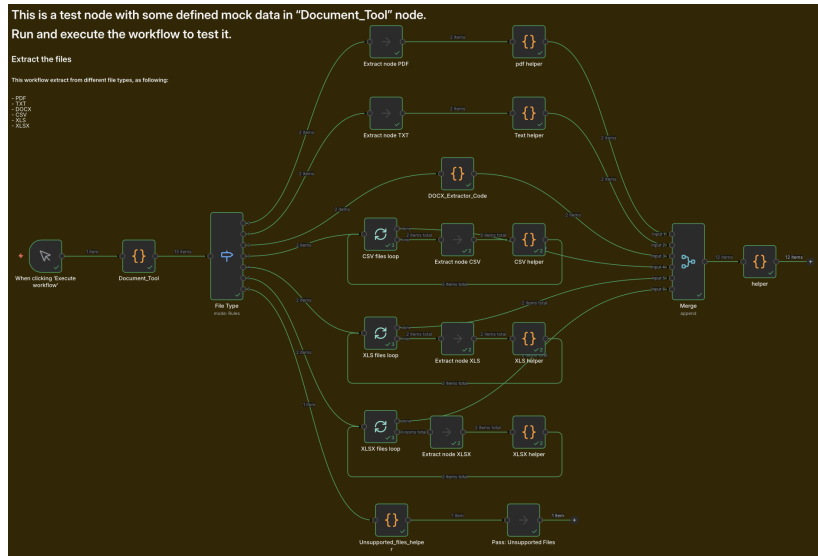


Figure 4.2: Example test of the File Extraction Module.

All automated functional tests produced expected flows, confirming that the logical paths of the system operate correctly under controlled conditions.

4.5.2 Manual End-to-End Testing

Following automated module verification, each agent was manually tested through the Chat Trigger interface using representative documents covering all supported file formats. This confirmed that data passed correctly between the extraction pipeline, the Orchestrator, and each specialised agent under realistic operating conditions, and that agent responses were returned to the chat interface without silent failures.

Manual testing was not used to assess output accuracy. Findings about the quality of summaries, the correctness of classifications, and the precision of extraction results are reserved for the experimental evaluation presented in Chapter 5.

4.6 Deployment and Reproducibility

The system is containerised using Docker Compose, encapsulating n8n and PostgreSQL as isolated services connected over a dedicated bridge network. Deployment requires no manual dependency installation beyond Docker itself. All external API credentials (specifically the Google AI Studio and Groq API keys) are configured through a single `.env` file prior to launch. An initialisation script (`initialise_project.sh`) automates the remaining setup: generating encryption keys, starting all containers, and importing the project workflows directly into n8n from the bundled `n8n_workflows/` directory.

Once running, the n8n interface is accessible at `http://localhost:5678`, with all workflows pre-loaded and ready for use. When initialised for the first time, two automatic values are generated `N8N_ENCRYPTION_KEY` & `N8N_API_KEY`; these should never be changed after initialisation as modifying either of these values will cause corruption of stored credentials. Full step-by-step setup instructions, prerequisite requirements per operating system, and service port references are documented in the project `README.md`, found in the project's directory.

```
services:
  postgres:
    ...
  n8n:
    ports:
```

```

    - '5678:5678'
    ...
volumes:
  n8n_data:
    driver: local
  postgres_data:
    driver: local

```

Listing 4.5: Docker Compose Configuration (excerpt)

4.7 Summary

This chapter explained the full implementation of AWOS as a unified n8n workflow realising the three-tier architecture defined in Chapter 3. The technology stack was selected on the basis of cost-efficiency and functional fit, with a hybrid inference strategy combining Google AI Studio and Groq LPU to balance context capacity against throughput. Three engineering challenges were resolved during development: semantic loss in long-context summarisation, non-deterministic routing output, and the tension between auditability and statelessness. Functional correctness was verified through automated Manual Trigger testing and manual end-to-end validation. The quantitative evaluation of agent output quality against the NFR targets is presented in Chapter 5.

Chapter 5

Evaluation

This chapter evaluates the Autonomous Workflow Orchestration System (AWOS) against the requirements established in Chapter 3 and the design claims made throughout this thesis. It begins by defining five research questions and the experimental setup used to answer them, presents the empirical results across all agents and logical components, and concludes with a discussion of findings, a comparison against related literature, identified limitations, and threats to validity.

5.1 Research Questions (RQ)

The evaluation is structured around five research questions, each related to a specific design claim or system requirement:

- **RQ1 Routing Correctness:** Does the Orchestration Agent correctly route natural language user requests to the appropriate specialised agent? (*Tests NFR-06 and the directed orchestration design.*)
- **RQ2 Task Performance:** How accurately do the three specialised agents (Summarisation, Classification, Data Extraction) perform their assigned tasks across the supported document formats? (*Tests NFR-02, NFR-03, and the core functional requirements FR-04, FR-06, FR-08.*)
- **RQ3 Overall Performance and Responsiveness:** Does the system meet its responsiveness targets under realistic document workloads? (*Tests NFR-01, NFR-04, NFR-11.*)
- **RQ4 Boundary Handling:** Does the Multi-Level Task Refusal pattern correctly iden-

tify and reject out-of-scope requests while preserving valid ones? (*Tests refusal flows introduced in Chapter 3 and AF-2A.*)

- **RQ5 Token Efficiency:** Which model architecture provides the best trade-off between reasoning accuracy and token consumption? (*Validates operational cost-efficiency.*)

5.2 Experimental Setup

5.2.1 Dataset Construction

The evaluation utilises a comprehensive dataset curated to test the system across 150 summarisation tasks, 100 classification scenarios, and 200 data extraction cases. The dataset covers:

- **Multilingual Support:** Ingestion of various languages, including English and Arabic, to test the linguistic flexibility of the underlying LLM APIs.
- **File formats:** Native support verification for PDF, DOCX, TXT, CSV, XLS, and XLSX (FR-03).
- **Complexity levels:** Stratified across “Easy” (simple text), “Medium” (standard business layouts), and “Hard” (multi-column layouts, embedded tables, and dense insurance/policy terminology).

Dataset Composition

Test Set	Size	Purpose
Summarisation	150 docs	Scientific papers, insurance policies, resumes, manuals, and contracts.
Classification	100 docs	Mixed types including resumes, slides, scientific papers, general files, and spreadsheets.
Data Extraction	200 docs	High-volume verification of bills and invoices in different difficulty levels.
Routing Prompts	30 cases	Natural language intents to test the Orchestrator.
Refusal Test Cases	20 cases	Verification of boundary-handling logic.

Table 5.1: Evaluation Dataset Composition

Ground Truth and Reference Benchmarks

To validate system outputs, a reference benchmark was established through manual verification. Classification tasks were assigned categories based on a professional taxonomy, while extraction tasks utilised manually verified JSON templates containing expected values for key data fields.

The summarisation performance was compared to the reference summaries selected by the authors; the inherent subjectivity of this qualitative assessment is addressed as a limitation in Section 5.4.3.

Given the technical diversity of the evaluation documents, an LLM-assisted auditing phase was implemented to identify potential discrepancies. These indicators were subsequently reviewed against the weighted rubric to ensure that human oversight remained the final authority. This methodology mitigated potential evaluator bias and ensured consistent scoring across varied document domains.

5.2.2 Evaluation Metrics

The system is evaluated using weighted rubrics (0-100) to move beyond binary pass/fail results and capture the nuances of LLM performance.

Qualitative Scoring Rubrics

Each specialised agent is assessed against a specific weighted rubric:

Agent	Criterion	Weight	Description
Summarisation	Coverage	30%	Presence of essential thematic key points from the source document.
	Accuracy	30%	Freedom from hallucinations, factual misrepresentation and precise explanation.
	Conciseness	20%	Efficiency of output without unnecessary padding.
	Clarity	20%	Readability and structural coherence.
Classification	Accuracy	40%	Correctness of <code>document.type</code> and <code>category</code> .
	Justification	30%	Alignment between content and the “info” field.
	Specificity	30%	Precision of labels rather than vague tags.
Data Extraction	Completeness	40%	Capture rate of all present extractable fields across the schema.
	Accuracy	40%	Exact value matching between extracted fields and ground truth.
	Structure	20%	Proper mapping of values to the defined schema.

Table 5.2: Weighted Scoring Rubrics for Specialised Agents

Performance Metrics

In addition to qualitative scoring, the following operational metrics were recorded to address RQ3 and RQ5:

- **End-to-end Execution Time:** The total elapsed time from workflow trigger to rendered output, measured per batch and reported in MM:SS format. Includes mandatory wait-node delays introduced to respect provider rate limits.
- **Token Consumption:** The total number of tokens consumed per agent across the full test set, used as the denominator in the Efficiency Score calculation.

- **Failure Rate:** The proportion of tested documents that produced an unrecoverable workflow error or no usable output, recorded separately for batch and sequential processing conditions.

5.2.3 Test Environment

The AWOS operational layer is containerised via Docker Compose, utilising n8n and PostgreSQL for workflow orchestration. As reasoning tasks were delegated to cloud-based APIs, local hardware specifications were a secondary performance variable.

Experimental variables included **Gemini 3.0 Flash** for Summarisation, **Gemini 3.1 Flash Lite** for both Classification and Extraction, **gpt-oss-120b**, and **llama-4-scout-17b-16e-instruct**. To ensure logic-gate consistency, the Orchestrator utilised deterministic response settings, while specialised agents allowed for standard linguistic variation. Performance data was extracted directly from n8n execution history logs.

Reported execution times include mandatory wait-node delays required to respect API rate limits. Furthermore, the results reflect a significant architectural disparity: the Gemini models utilised large context windows to process document batches in a single request, whereas GPT-OSS and LLAMA 4 required serial, file-by-file processing to maintain stability. These differences in throughput are further analysed in Section 5.4.

5.3 Results

This section quantifies the system’s performance across the five primary research questions. To facilitate a clear comparison of the hybrid architecture, results are consolidated into comparative tables evaluating logical accuracy, agent precision, and operational efficiency.

To provide a deterministic measure of performance that accounts for resource utilisation, an **Efficiency Score** is calculated for each task. This metric represents the intelligence density of a model by measuring the accuracy achieved per 100,000 tokens consumed:

$$\text{Efficiency Score} = \left(\frac{\text{Weighted Average Accuracy (\%)}}{\text{Total Tokens Consumed}} \right) \times 10^5 \quad (5.1)$$

By utilising this ratio, the evaluation identifies which model architecture provides the optimal trade-off between reasoning quality and operational cost, providing a measurement that is independent of variable network latencies.

5.3.1 Logical Integrity: Routing and Refusal (RQ1 & RQ4)

The system’s logical pipelines were tested to ensure the Orchestrator correctly handles both valid and out-of-scope requests. As routing correctness is a binary logical operation rather than a token-dependent task, the Efficiency Score metric is not applicable to RQ1 and RQ4. The Orchestrator was evaluated using both Gemini 2.5 Flash and Qwen 3. Both models produced comparable routing accuracy; however, Qwen 3 was selected for production use due to its higher API call allowance under the free tier, offering greater throughput headroom for the orchestration layer.

Logical Operation	Total Cases	Correct Decisions	Accuracy (%)
Intent Routing (RQ1)	30	29	96.7%
Level 1 Refusal (Out-of-Scope)	10	10	100.0%
Level 2 Refusal (In-Agent)	10	8	80.0%
System Logical Mean	50	47	94.0%

Table 5.3: Functional Accuracy of Orchestration and Boundary Handling

5.3.2 Summarisation Agent Performance (150 Files)

The summarisation results demonstrate a significant disparity in reasoning efficiency, as measured by the Efficiency Score introduced above. Despite similar token footprints across the three architectures, the semantic quality of the outputs varied considerably. Gemini 3.0 Flash achieved the highest performance across all metrics in this category, completing the 150-file workload in 14:22, substantially faster than both alternative models.

Metric	Gemini 3.0 Flash	GPT-OSS	LLAMA 4
Weighted Avg. Accuracy	89.0%	68.0%	52.0%
Total Execution Time (MM:SS)	14:22	57:34	58:05
Total Tokens Consumed	397,661	402,560	402,825
Efficiency Score	22.38	16.89	12.90

Table 5.4: Summarisation Reasoning Efficiency (RQ2 & RQ5)

5.3.3 Classification Agent Performance (100 Files)

The classification evaluation focuses on the precision of intent and category assignment. As shown in Table 5.5, GPT-OSS achieved the highest accuracy (95.0%) and Efficiency Score (71.00), demonstrating greater label specificity than the other models. This positions it as the optimal model for classification tasks despite its longer execution time.

Metric	Gemini 3.1 Flash Lite	GPT-OSS	LLAMA 4
Weighted Avg. Accuracy	83.5%	95.0%	58.0%
Total Execution Time (MM:SS)	5:42	32:46	30:02
Total Tokens Consumed	131,746	133,794	129,073
Efficiency Score	63.40	71.00	44.94

Table 5.5: Classification Reasoning Efficiency (RQ2 & RQ5)

5.3.4 Data Extraction Agent Performance (200 Files)

The data extraction task measured the models’ ability to map precise field values to a best-fit schema. Gemini 3.1 Flash Lite significantly outperformed the other models in efficiency, processing 200 files in less than 8 minutes with the highest weighted accuracy of 85.0%.

Metric	Gemini 3.1 Flash Lite	GPT-OSS	LLAMA 4
Weighted Avg. Accuracy	85.0%	71.5%	53.0%
Total Execution Time (MM:SS)	7:25	62:18	58:47
Total Tokens Consumed	252,508	400,847	310,489
Efficiency Score	33.66	17.84	17.07

Table 5.6: Data Extraction Performance and Efficiency Comparison (RQ2 & RQ5)

GPT-OSS and LLAMA 4 were evaluated using sequential single-file API calls. Batch submission was attempted, but produced critical failure rates exceeding 70% for both models. Failure analysis indicated that under multi-document prompts, both models exhibited severe hallucination and task abandonment, generating responses unrelated to the submitted content rather than processing each document independently. Reverting to single-file submission resolved these failures, suggesting the behaviour stems from context-window coherence degradation rather than a fundamental incapacity for the task. This constraint accounts for the substantially higher token consumption and execution times relative to Gemini, which processed documents in larger batches without degradation.

5.4 Discussion

Experimental results indicate that AWOS has met three of the four key NFRs. The routing accuracy was 96.7% (NFR-06, target value $\geq 95\%$), classification precision reached 95.0% (NFR-02, target $> 85\%$), and boundary handling achieved a system logical mean of 94.0%. Level 1 refusals operate on the Orchestrator and target out-of-scope intents prior to invoking any agent. Level 2 refusals (AF-2A) operate on each individual specialised agent, reject documents that

do not fit the intended task of the agent, and return a structurally justified response to the user. The extraction precision achieved 85.0% compared to the target of 90% (NFR-03), or five percentage points less than expected. The factors contributing to this difference and their implications are detailed in the section that follows.

5.4.1 Interpreting Results

Routing Performance (RQ1)

The Orchestration Agent achieved a routing accuracy of 96.7%, validating the directed orchestration design adopted in this work. This result aligns with findings reported by Klang et al.[26], who demonstrated that multi-agent systems maintained high routing accuracy (90.6%) across five concurrent tasks, while single-agent systems experienced significant performance collapse. By sustaining accurate routing across three specialised agents alongside boundary handling logic, AWOS substantiates the position that modular multi-agent architectures exhibit greater resilience than monolithic alternatives. This precision is consistent with the two-layer constraint strategy described in Chapter 4, whereby deterministic generation parameters combined with a Structured Output Parser bounded model output variability, translating stochastic LLM responses into deterministic routing decisions.

Task Performance (RQ2)

The performance gap between classification (95.0%) and data extraction (85.0%) provides insight into both the capabilities and current limitations of general-purpose LLMs for document processing tasks.

The 95.0% classification accuracy achieved by GPT-OSS suggests that large-parameter models demonstrate sufficient capacity to identify subtle semantic distinctions within documents containing complex vocabulary and structure, such as legal contracts and resumes. By contrast, extraction precision reached only 85.0%, falling short of the NFR-03 target by five percentage points. Analysis of failed cases reveals that degradation was predominantly localised to “Hard” complexity documents characterised by irregular multi-column layouts and overlapping tabular structures. This indicates that general-purpose LLMs without domain-specific fine-tuning struggle when document structure, rather than linguistic content, determines the location of extractable fields. This represents a recognised boundary of the current system and is discussed further in Section 5.4.3.

Performance and Response Time (RQ3 & RQ5)

The results reveal a fundamental trade-off between large context window efficiency and the overhead of serial, file-by-file processing.

Gemini 3.0 Flash processed the 150-document summarisation dataset in 14:22, affirming the strategic value of large context window architectures for high-volume ingestion. The speed advantage over GPT-OSS (57:34) reflects not merely faster API response times but a structural efficiency gain: Gemini’s capacity to ingest multiple documents within a single context window eliminates the cumulative latency introduced by the repeated prompt initialisation required for serial, file-by-file processing.

With respect to NFR-01, the per-document processing time under Gemini remained well within the 30-second target across all three agents. GPT-OSS and LLAMA 4 exceeded this threshold consistently due to serial processing constraints, reinforcing Gemini as the production-viable model for latency-sensitive workloads.

The Efficiency Score for RQ5 clearly shows that although GPT-OSS possesses the highest reasoning efficiency during classification operations, Gemini is ultimately the most operationally viable model for use in the integrated system overall. This supports the proposed hybrid selection strategy of selecting high precision models for independent classification operations and high efficiency models for bulk summarisations and extractions.

Boundary Handling (RQ4)

The Multi-Level Task Refusal pattern achieved perfect accuracy at Level 1, successfully rejecting all out-of-scope prompts before any specialised agent was invoked. However, Level 2 accuracy fell to 80.0%, indicating a behavioural tendency toward over-helpfulness in specialised agents when presented with ambiguous inputs. In specific cases, agents attempted to partially fulfill requests that should have been rejected outright. While the external orchestration gates perform robustly, this finding suggests that internal system prompts for specialised agents require stricter negative constraints to prevent agents from inferring utility from out-of-scope content.

5.4.2 Comparison with Related Work

The results produced by AWOS are interpreted against two bodies of literature reviewed in Chapter 2: empirical benchmarks for LLM task performance (Section 2.5), and theoretical claims regarding multi-agent orchestration architectures (Section 2.6). It is important to note that all results reported in this evaluation reflect the specific models available under free-tier

API access at the time of testing. Higher-capability and newer models including Claude 4.6 Sonnet, GPT-4o, GPT-5 and Gemini 3.1 Pro were excluded from evaluation due to limited access. Therefore, the upper limit of this study represents an approximate performance ceiling and does not necessarily represent the potential capability of the AWOS architecture with better or higher-capable models selected.

LLM Task Performance Against Literature Benchmarks

Section 2.5 provided literature evidence of baseline performance levels associated with LLM-based task completion for premium models. Hsu et al. have found high-quality extraction rates ($> 92\%$) when utilizing GPT-4o and Gemini-1.5-Pro on clinical documents [22]. Similarly, Hassani et al. demonstrated that GPT-4o provided meaningful improvements over legacy models for classifying complex regulatory language [19]. Saxena et al. demonstrated that summaries produced by Llama 3 and GPT-3.5 exhibited greater contextual fidelity and structural clarity than other comparative models [39].

In general, the AWOS results provide similar conclusions to those studies cited above; however, AWOS utilised free tier models. As such, classification was 95.0% accurate with GPT-OSS-120b. This result corresponds to the literature’s depiction of well-performing large parameter models being best suited for taxonomic-type tasks requiring very fine-grained distinction among semantically-related concepts. Additionally, summarisation results were 89.0% accurate when using Gemini 3.0 Flash. No direct summarisation benchmark at equivalent dataset scale was identified in the literature reviewed; however, this result is consistent with the broader finding that newer generation models perform better than previous models at creating contextually coherent summaries. Extraction precision results fell slightly short of both the NFR-03 target and the 92% threshold reported by Hsu et al. [22]. This gap is attributable to two factors. First, the clinical documents evaluated by Hsu et al. are structurally consistent [22], whereas the AWOS extraction dataset included irregular multi-column layouts and overlapping tabular structures that general-purpose models handle poorly without domain-specific fine-tuning. Second, the models available under free-tier constraints are less capable than GPT-4o and Gemini-1.5-Pro, which were used to achieve the literature benchmark. These two factors together account for the observed performance gap and suggest the gap would narrow substantially under equivalent model access.

Multi-Agent Stability and Context Interference

Klang et al. reported that at five concurrent tasks, multi-agent systems achieved 90.6% accuracy compared to 73.1% for single-agent systems. Under heavier load at 80 tasks, this gap widened substantially, with multi-agent systems maintaining 65.3% accuracy while single-agent performance collapsed to 16.6% [26]. AWOS maintained high routing accuracy of 96.7% across three specialised agents, consistent with the broader finding by Klang et al. that modular multi-agent architectures sustain reliable performance under concurrent task load, in contrast to the collapse observed in single-agent systems [26]. By delegating tasks to isolated specialised agents, the system maintained high accuracy across summarisation 89.0% and classification 95.0% without the accuracy collapse typical of monolithic prompts. This substantiates the claim made by Klang et al. that worker-agent isolation insulates the model from context interference. Furthermore, the batch failures observed in GPT-OSS and LLAMA 4 during extraction testing serve as direct evidence of this phenomenon. When multiple documents were submitted within a single prompt, both models exhibited hallucination and task abandonment, which aligns with the degradation Klang et al. attributed to context overload in non-orchestrated architectures.

Orchestration as a Design Requirement

Masters et al. identified centralised orchestration as an open challenge, arguing that a managerial coordination layer is necessary to prevent logical contradictions in multi-agent workflows [31]. Adimulam et al. further stated that such a layer enables independent agents to operate as a unified, purpose-driven system [3]. The AWOS results provide practical support for both claims. The Orchestration Agent produced zero logical contradictions across all 30 routing test cases, with the single routing failure attributable to an ambiguous natural language prompt rather than a structural fault in the orchestration layer. Importantly, this was achieved through a directed, rule-bounded orchestrator rather than a fully autonomous manager agent, suggesting that the level of orchestration complexity described by Masters et al. as an open challenge is not a prerequisite for reliable routing in single-user, bounded-task deployments.

Auditability as an Architectural Property

Klang et al. identified auditability as a direct benefit of the worker-agent constraint model, noting its importance for regulatory compliance and workflow reproducibility [26]. Hsu et al. reported that GPT-4o and Gemini-1.5-Pro demonstrated reproducibility rates approaching 99%

make these models suitable for integration into real-time systems [22]. AWOS manifests auditability through Metadata-Content Decoupling, whereby PostgreSQL stores execution metadata including routing decisions, timestamps, and agent selections, while document content remains exclusively in volatile memory. This satisfies NFR-12 without compromising NFR-09, demonstrating that auditability and statelessness are not mutually exclusive properties in lightweight single-user deployments. This nuance is not explicitly addressed in the enterprise-focused systems surveyed in the literature, where auditability typically assumes persistent data storage.

5.4.3 Limitations

Although the AWOS demonstrated operational feasibility as a lightweight multi-agent document processing application, there exist several limitations regarding scalability, applicability and usability. Where relevant, these limitations are reflected in the interpretation of results presented in Section 5.4.

Free-Tier API Constraints

The system’s reliance on free-tier API access imposes token-per-minute rate limits that directly bound the volume of documents processable within a single session. The amount of documents that can be processed in one session is directly dependent on the token-per-minute limit. Furthermore, the chat history agent is constrained by available context window capacity, which limits the depth of conversational history memory maintained across extended sessions. These constraints are a product of the cost-efficiency design goals rather than architectural deficiencies; however, they represent a practical barrier to production deployment at scale.

Stateless Document Storage

In compliance with NFR-09, all documents provided by users have no state persistence after termination of the active session. Therefore, documents previously processed by AWOS are unavailable for future reference or re-query unless they are uploaded again. Such constraints create a serious limitation on utilising AWOS for workflows where it is desirable to maintain access to previous versions of documents used in a workflow and/or cross-session continuity.

Absence of OCR and Visual Content Parsing

The system operates exclusively on text extracted from uploaded files and lacks the capability to interpret embedded images, figures, or visually rendered content. For real-world document types such as utility bills and consumption reports, critical data is frequently encoded graphically rather than as extractable text. As a result, the extraction agent processes only the textual portion of a document and may miss operationally important information. Addressing this gap would require adding an Optical Character Recognition (OCR) or vision-language model component to the system.

Spreadsheet Data Aggregation

The agents examined within the AWOS framework demonstrate poor capabilities in analysing spreadsheets including CSV and XLSX. A major issue exists in tasks which involve multi-row data aggregation and accuracy in mathematical consistency. The ingestion of these file formats by the system is possible but the models utilised often view tabular data as unstructured flat text resulting in many errors relating to referencing cells based on spatial location and tabular reasoning. Therefore, AWOS is currently incapable of providing accurate financial auditing or performing complex data manipulation.

Extraction and Summarisation Reliability

LLMs are generally prone to hallucination and field-level confusion when extracting structured data fields, particularly from documents containing duplicate fields, ambiguous formatting, or domain-specific terminology. The data extraction agent is significantly impacted by this limitation because errors in numerical and reference fields are typically non-recoverable compared to those found in qualitative tasks. Since no validation layer is included to validate extracted values against expected schema, such errors may be propagated to the end user without warning.

Single-User Scope and Concurrency

AWOS was designed and evaluated exclusively within a single-user context. The system has not been tested under concurrent load, and the current architecture lacks mechanisms for session isolation or request queuing across multiple simultaneous users. Deployment in a multi-user environment would require significant architectural extensions to manage shared resources and PostgreSQL transaction locks.

Third-Party API Dependency

All inference tasks are delegated to external cloud-based APIs. System availability and performance are therefore subject to provider up-time, rate-limit policy changes, and model version updates. As LLM providers update their models continuously, results reported in this evaluation may not be fully reproducible in future runs, even under identical prompting conditions.

Evaluation Subjectivity

A subjective metric is used in evaluating the summary quality metric since it relies on reference summaries created by researchers who scored summaries based on a weighted metric. Although an LLM-assisted auditing phase has been added to reduce subjectivity, qualitative assessments inherently possess some degree of subjectivity. Therefore, inter-rater agreement cannot be determined. Consequently, the results obtained by the summarisation agent should be interpreted as indicative rather than definitive.

Sequential Processing Constraint

As discussed in the results section, GPT-OSS and LLAMA 4 were evaluated under sequential single-file conditions due to batch failure rates exceeding 70%, as reported in Section 5.3. Throughput comparisons between models are therefore constrained, as sequential processing represents a significantly reduced operational mode relative to realistic high-volume batch submission scenarios.

5.4.4 Threats to Validity

- **Internal Validity:** Results may be influenced by factors beyond model capability, including prompt wording, wait-node timing, and API response variability on the day of testing. No ablation tests were conducted to isolate the contribution of individual design decisions.
- **External Validity:** The evaluation dataset is limited to specific document domains and two languages. As such, results may not generalise to document types, languages, or professional contexts not represented in the test set.
- **Construct Validity:** The Efficiency score represents accuracy per token inputted, however, it does not take into account the number of tokens outputted, the complexity of the prompts inputted by users to receive responses, nor the time delay in receiving the

response from the APIs. The Efficiency Score should therefore be interpreted as a relative comparative metric rather than an absolute measure of operational cost.

- **Reliability:** LLM providers update deployed models continuously without versioned guarantees on free-tier endpoints. Results reported here may not be fully reproducible on a future run, even under identical prompting conditions and dataset inputs.

5.5 Summary

This chapter evaluated AWOS against its defined requirements through a structured experimental design covering five research questions. Three of the four primary NFRs were satisfied: routing accuracy reached 96.7% (NFR-06), classification precision reached 95.0% (NFR-02), and boundary handling achieved a system logical mean of 94.0%. Extraction precision reached 85.0%, falling short of the 90% target (NFR-03), with failures concentrated in structurally complex documents. The hybrid model strategy was validated, with Gemini models demonstrating superior throughput efficiency and GPT-OSS achieving the highest classification accuracy. Results were interpreted against literature benchmarks and architectural claims, with the multi-agent design substantiating the resilience advantages reported by Klang et al. Key limitations include free-tier throughput constraints, the absence of OCR capability, and evaluation subjectivity in summarisation scoring. The findings and limitations identified here inform the conclusions and future work presented in Chapter 7.

Chapter 6

Legal, Social, Ethical and Professional Issues

This chapter examines the legal, social, ethical, and professional considerations arising from the development and deployment of AWOS. It demonstrates awareness of the regulatory frameworks governing AI-based document processing systems and reflects on how the principles of the BCS Code of Conduct were applied throughout the project.

6.1 Legal Considerations

6.1.1 Data Protection and GDPR

The General Data Protection Regulation (GDPR) establishes legally enforceable obligations for the processing of personal data [35]. Documents processed by AWOS, including utility bills, invoices, and contracts, frequently contain Personally Identifiable Information (PII) such as names, addresses, and financial records. NFR-09 addresses this directly, as all user-uploaded content is retained exclusively in volatile memory and purged upon session termination, satisfying the GDPR principle of “Data Minimisation” [35]. Since document content is never written to non-volatile storage, the “Right to Erasure” [35] is inherently satisfied without requiring an explicit deletion mechanism.

A residual concern involves transmission of document content to third-party APIs. Both Google AI Studio [17] and Groq [18] state that subscriber data is not used for model training; however, this represents a dependency on third-party compliance outside the developer’s

control. In deployments involving sensitive data, field-level encryption or “pseudonymisation” prior to transmission would be a necessary extension [35].

Beyond Data Minimisation and the Right to Erasure, three further GDPR principles are satisfied by design. “Lawfulness” requires a valid legal basis for processing [35]; AWOS processes documents on the basis of the user’s explicit request, constituting a clear legitimate purpose. “Purpose Limitation” requires that data be processed only for the specific purpose for which it was collected [35]; document content is used exclusively for the user-requested task and discarded immediately upon completion. “Accountability” requires that controllers demonstrate compliance [35]; this is satisfied through the PostgreSQL execution audit trail (NFR-12), which records routing decisions, timestamps, and agent selections for every workflow execution. It should be noted that accountability at the inference layer remains dependent on third-party provider compliance, as the system has no visibility into how document content is handled within external API infrastructure.

6.1.2 AI Regulation and Compliance

The EU Artificial Intelligence Act classifies AI systems according to risk level [2]. Risk classification is deployment-dependent rather than fixed at the architectural level. In its primary administrative context, processing invoices, contracts, and technical reports, AWOS falls under the minimal-risk category, requiring transparency obligations rather than conformity assessments [2]. However, certain document types within the supported classification use case, such as resumes and employment-related records, could place the system in the high-risk category if used to inform hiring decisions, as the Act explicitly identifies AI used in employment screening as high-risk [2]. This distinction places an obligation on deploying organisations to assess the intended use context before deployment. For administrative use cases, transparency obligations are satisfied through the PostgreSQL execution audit trail (NFR-12).

6.1.3 Security Considerations

The system manages API credentials through environment variables isolated from workflow logic and excluded from version control. Container network isolation via Docker Compose restricts inter-service communication to defined ports. Prompt injection, where adversarial content embedded in an uploaded document attempts to override system instructions, is partially mitigated by the Structured Output Parser constraining Orchestrator output to a fixed set of intent labels. Specialised agents that process document content directly remain a residual

attack surface in adversarial deployment contexts.

6.2 Ethical Considerations

6.2.1 Transparency and User Trust

A core ethical requirement for automated systems is that users understand what their data is being used for by the system, as transparency is essential to generate trust [9]. AWOS addresses this through inspectable routing decisions visible in the n8n execution log and structured human-readable outputs that allow users to verify agent decisions, ensuring the processing logic remains intelligible [9]. The stateless design of the system also precludes sensitive content from accumulating within the system over time, addressing the common big data tendency to collect and retain as much data as possible [9]. However, third-party LLM API dependencies require users to trust that provider infrastructure handles their data responsibly; this highlights a persistent knowledge gap and imbalance of power that is difficult for users to verify independently [9].

6.2.2 Bias and Fairness

General-purpose LLMs inherit biases present in their training data, which can affect classification outcomes in ways that are difficult to detect [9]. For AWOS, this is most relevant to the Classification Agent. When classifying documents such as resumes, the underlying model may reflect demographic or linguistic biases present in its training corpus, potentially producing different classification outcomes for structurally equivalent documents from different cultural or linguistic contexts. The multilingual evaluation conducted in this project, covering English and Arabic documents, provides partial evidence of cross-linguistic performance, but does not constitute a systematic bias audit. Organisations deploying AWOS for document classification in sensitive professional contexts should conduct domain-specific bias testing before relying on classification outputs for consequential decisions.

6.3 Social Impact

6.3.1 Automation and Workforce Implications

AWOS automates document processing tasks currently performed manually in administrative roles. Automating repetitive, rule-based processes reduces manual intervention and labour

costs while improving consistency and operational efficiency [51]. The appropriate framing is augmentation rather than replacement: AI decisioning functions as a decision-support tool, enabling professionals to redirect effort toward higher-value tasks requiring contextual judgement and domain expertise [51].

6.3.2 Accessibility

Traditional AI development demands high costs and specialised expertise, creating a significant barrier to entry for many organisations [51]. Low-code and no-code platforms lower this barrier by providing cost-effective, simplified deployment approaches, enabling small and medium-sized enterprises (SMEs) to leverage machine learning without dedicated AI teams [51]. AWOS exemplifies this directly: by combining free-tier API access with open-source orchestration tooling, it demonstrates that capable multi-agent document processing is achievable without enterprise-scale investment.

6.4 Professional Considerations

6.4.1 Implications for Computer Science Practice

The development of AWOS reflects a broader shift toward low-code and AI-native architectures in software engineering. As platforms such as n8n abstract infrastructure concerns, the role of the computing professional is evolving from writing procedural code to designing, auditing, and governing AI-driven workflows [51]. This shift introduces new professional responsibilities: engineers must critically evaluate vendor reliability, API stability, and the auditability of automated decisions rather than simply implementing functional requirements. The BCS Code of Conduct reinforces this obligation, requiring members to maintain awareness of technological developments and carry out responsibilities with due care and diligence [16]. The deliberate rejection of more autonomous frameworks in favour of directed orchestration reflects a professional judgement that auditability and predictability should take precedence over capability when handling sensitive user data [16].

6.4.2 Implications for Other Professions

AWOS is designed for use across any professional domain that relies heavily on document processing. Legal professionals handling contracts, financial analysts processing invoices, insurance adjusters reviewing policy documents, and healthcare administrators managing clinical records

could all benefit from automated summarisation, classification, and data extraction. For these professionals, AWOS functions as a productivity tool that reduces time spent on document review and allows focus on higher-order professional judgement.

However, users in regulated professions must exercise caution: outputs produced by general-purpose LLMs should be treated as decision-support rather than authoritative conclusions, and professional accountability for the final decision remains with the practitioner. The BCS Code of Conduct recognises this principle, noting that computing professionals have a duty to protect third parties affected by their systems and to promote access to the benefits of IT across all sectors of society [16]. The PostgreSQL execution audit trail produced by AWOS supports this accountability requirement by providing a verifiable record of which agent processed each request and when, enabling professionals to demonstrate due diligence in their use of automated decision-support tools.

6.4.3 Adherence to Professional Standards

The BCS Code of Conduct requires computing professionals to act in the public interest, maintain competence, and behave with integrity [16]. The privacy-by-design approach embodied in NFR-09 prioritises public interest over system convenience, consistent with the Code’s requirement to have due regard for privacy and the wellbeing of others [16]. Honest reporting of NFR-03 non-achievement reflects the professional obligation not to misrepresent the performance of systems [16]. The use of versioned, exportable workflow assets ensures the system remains maintainable and auditable by parties other than the original developer, consistent with BCS obligations around knowledge sharing and professional accountability [16].

Chapter 7

Conclusion and Future Work

7.1 Conclusion

This project presented the design, implement, and evaluate a multi-agent document processing system grounded in directed orchestration, as a cost-efficient alternative to fully autonomous frameworks. AWOS was developed as a three-tier n8n workflow supporting summarisation, classification, and data extraction across five file formats, deployed entirely on free-tier API infrastructure.

All six project objectives were met. A hierarchical orchestration architecture was designed and implemented, multi-format ingestion was achieved through a combination of native nodes and a custom Mammoth-based extraction engine, and a Multi-Level Task Refusal mechanism was validated at both the orchestration and agent layers. The system was deployed reproducibly via Docker Compose, evaluated against defined non-functional requirements, and a three-model empirical comparison was conducted using the proposed Efficiency Score metric.

Three of four primary NFRs were satisfied. Routing accuracy reached 96.7% (NFR-06), classification precision reached 95.0% (NFR-02), and boundary handling achieved a system logical mean of 94.0%. Extraction precision reached 85.0%, falling short of the 90% target (NFR-03), with failures concentrated in structurally complex documents. The hybrid model strategy was validated, with Gemini models demonstrating superior throughput and GPT-OSS achieving the highest classification accuracy.

The central research question of whether directed orchestration can deliver reliable document processing without the complexity of autonomous frameworks is answered affirmatively. AWOS demonstrated that a rule-bounded, cost-efficient architecture achieves routing accuracy

exceeding published multi-agent benchmarks [26], without autonomous self-direction, iterative re-planning, or enterprise-scale API expenditure.

Key limitations include the reliance on free-tier API constraints, the absence of OCR capability, and the evaluation subjectivity inherent in summarisation scoring. NFR-04 and NFR-11 were not formally evaluated at their full defined thresholds due to rate-limit constraints. These are discussed in detail in Section 5.4.3.

7.2 Future Work

Several directions emerge directly from the findings and limitations of this project:

- **OCR and vision-language integration:** Adding an OCR or vision-language model component would address the critical gap identified in graphically rendered documents such as utility bills.
- **Spreadsheet reasoning:** Although spreadsheet files are extracted and passed to agents as structured JSON, current general-purpose LLMs struggle to perform accurate multi-row aggregation and mathematical reasoning over tabular data. A future implementation could incorporate a dedicated code execution layer, such as a Python sandbox, to handle numerical operations programmatically rather than delegating them to the LLM, significantly improving accuracy for financial and analytical spreadsheet tasks.
- **Premium model evaluation:** Testing AWOS with GPT-5.4 Pro, Claude Sonnet 4.6 and Opus 4.7, and Gemini 3.1 Pro would establish whether the NFR-03 extraction gap closes under equivalent model access to the literature benchmarks.
- **Multi-user deployment:** Extending the architecture with session isolation and request queuing would enable concurrent use, addressing the single-user scope identified as a limitation.
- **Field-level encryption:** Implementing pseudonymisation prior to API transmission would strengthen the privacy guarantees identified in the Legal chapter for sensitive document deployments.

References

- [1] Mohamad Abou Ali, Fadi Dornaika, and Jinan Charafeddine. Agentic ai: a comprehensive survey of architectures, applications, and future directions. *Artificial Intelligence Review*, 59(1):11, 2025.
- [2] Artificial Intelligence Act. Artificial intelligence act. *ArtificialIntelligenceAct.eu*, <https://artificialintelligenceact.eu>, 2024.
- [3] Apoorva Adimulam, Rajesh Gupta, and Sumit Kumar. The orchestration of multi-agent systems: Architectures, protocols, and enterprise adoption. *arXiv preprint arXiv:2601.13671*, 2026.
- [4] Avinash Reddy Aitha. The evolution from automation to agentic ai. *Available at SSRN 5590590*, 2025.
- [5] Ajay Bandi, Bhavani Kongari, Roshini Naguru, Sahitya Pasnoor, and Sri Vidya Vilipala. The rise of agentic ai: A review of definitions, frameworks, architectures, applications, evaluation metrics, and challenges. *Future Internet*, 17(9):404, 2025.
- [6] Vicente J Bermejo, Andrés Gago, Ramiro H Gálvez, and Nicolás Harari. Llms outperform outsourced human coders on complex textual analysis. *Scientific Reports*, 15(1):40122, 2025.
- [7] Michael James Bommarito, Jillian Bommarito, and Daniel Martin Katz. What is an agent? a conceptual primer and history of agents and agentic ai. *A Conceptual Primer and History of Agents and Agentic AI (November 25, 2025)*, 2025.
- [8] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

- [9] Michael Butterworth. The ico and artificial intelligence: The role of fairness in the gdpr framework. *Computer Law & Security Review*, 34(2):257–268, 2018.
- [10] Zina Chkirbene, Ridha Hamila, Ala Gouissem, and Unal Devrim. Large language models (llm) in industry: A survey of applications, challenges, and trends. In *2024 IEEE 21st International Conference on Smart Communities: Improving Quality of Life using AI, Robotics and IoT (HONET)*, pages 229–234. IEEE, 2024.
- [11] Hayley Clatterbuck, Clinton Castro, and Arvo Muñoz Morán. Risk alignment in agentic ai systems. *arXiv e-prints*, pages arXiv–2410, 2024.
- [12] Yufan Dang, Chen Qian, Xueheng Luo, Jingru Fan, Zihao Xie, Ruijie Shi, Weize Chen, Cheng Yang, Xiaoyin Che, Ye Tian, et al. Multi-agent collaboration via evolving orchestration. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.
- [13] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186, 2019.
- [14] Luigi Gianpio Di Maggio. Predictive maintenance mcp: An open-source framework for bridging large language models and industrial condition monitoring via the model context protocol. *Applied Sciences*, 16(6):2812, 2026.
- [15] Shengda Fan, Xin Cong, Yuepeng Fu, Zhong Zhang, Shuyan Zhang, Yuanwei Liu, Yesai Wu, Yankai Lin, Zhiyuan Liu, and Maosong Sun. WorkflowLLM: Enhancing workflow orchestration capability of large language models. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [16] The Chartered Institute for IT. Bcs code of conduct. <https://www.bcs.org/membership-and-registrations/become-a-member/bcs-code-of-conduct/>, 2022.
- [17] Google. Google ai studio. <https://aistudio.google.com>, 2023.
- [18] Groq. Groq lpu. <https://console.groq.com>, 2024.
- [19] Shabnam Hassani, Mehrdad Sabetzadeh, and Daniel Amyot. An empirical study on llm-based classification of requirements-related provisions in food-safety regulations. *Empirical Software Engineering*, 30(3):72, 2025.

- [20] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. Metagpt: Meta programming for a multi-agent collaborative framework. In *The twelfth International Conference on Learning Representations, ICLR*, 2024.
- [21] Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. Model context protocol (mcp): Landscape, security threats, and future research directions. *ACM Transactions on Software Engineering and Methodology*, 2025.
- [22] Hsing-Yu Hsu, Lu-Wen Chen, Wan-Tseng Hsu, Yow-Wen Hsieh, and Shih-Sheng Chang. Extracting clinical guideline information using two large language models: Evaluation study. *Journal of Medical Internet Research*, 27:e73486, 2025.
- [23] Md Hasan Ibrahim, Mohammed Sayagh, and Ahmed E Hassan. A study of how docker compose is used to compose multi-component systems. *Empirical Software Engineering*, 26(6):128, 2021.
- [24] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM computing surveys*, 55(12):1–38, 2023.
- [25] Uday Kamath, Kevin Keenan, Garrett Somers, and Sarah Sorenson. Large language models: A deep dive. *Springer Nature, DOI*, 10:978–3, 2024.
- [26] Eyal Klang, Mahmud Omar, Ganesh Raut, Reem Agbareia, Prem Timsina, Robert Freeman, Nicholas Gavin, Lisa Stump, Alexander W Charney, Benjamin S Glicksberg, et al. Orchestrated multi agents sustain accuracy under clinical-scale workloads compared to a single agent. *npj Health Systems*, 3(1):23, 2026.
- [27] Chia-Hsuan Lee, Hao Cheng, and Mari Ostendorf. Orchestrallm: Efficient orchestration of language models for dialogue state tracking. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 1434–1445, 2024.
- [28] Guohao Li, Hasan Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. Camel: Communicative agents for” mind” exploration of large language model society. *Advances in neural information processing systems*, 36:51991–52008, 2023.

- [29] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM computing surveys*, 55(9):1–35, 2023.
- [30] Xiaoxia Liu, Peng Di, Cong Li, Jun Sun, and Jingyi Wang. Efficient function orchestration for large language models. *IEEE Transactions on Software Engineering*, 2025.
- [31] Charlie Masters and Stefano V Albrecht. Orchestrating human-ai teams: The manager agent as a unifying research challenge. In *LLM-based Multi-Agent Systems: Towards Responsible, Reliable, and Scalable Agentic Systems*, 2025.
- [32] Shakked Noy and Whitney Zhang. Experimental evidence on the productivity effects of generative artificial intelligence. *Science*, 381(6654):187–192, 2023.
- [33] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.
- [34] Adityamallikarjunkumar Parakala and Preston Padgett. When ai acts: Opportunities and risks of agentic systems. *International Journal of Artificial Intelligence, Data Science, and Machine Learning*, 6(4):29–40, 2025.
- [35] THE EUROPEAN PARLIAMEN. Regulation (eu) 2016/679 of the european parliament and of the council, general data protection regulation (gdpr). *Official Journal of the European Union*, 2016.
- [36] Anup Rao. Multi-agent ai orchestration using mcp and semantic kernel for autonomous enterprise systems. *Journal of Information Systems Engineering and Management*, 10:253–260, 2025.
- [37] Sumedh Rasal and EJ Hauer. Navigating complexity: Orchestrated problem solving with multi-agent llms. *arXiv preprint arXiv:2402.16713*, 2024.
- [38] Ranjan Sapkota, Konstantinos I Roumeliotis, and Manoj Karkee. Ai agents vs. agentic ai: A conceptual taxonomy, applications and challenges. *Information Fusion*, page 103599, 2025.

- [39] Rishabh Saxena, Shubhangi Singh, and Preeti Dubey. Comparative analysis of large language models for the application of scientific article summarization. *Procedia Computer Science*, 259:532–542, 2025.
- [40] Johannes Schneider. Generative to agentic ai: Survey, conceptualization, and challenges. *arXiv e-prints*, pages arXiv–2504, 2025.
- [41] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652, 2023.
- [42] Wei Song, Haonan Zhong, Ziqi Ding, Jingling Xue, and Yuekang Li. Help or hurdle? rethinking model context protocol-augmented large language models. *arXiv preprint arXiv:2508.12566*, 2025.
- [43] Michael Stonebraker, Lawrence A Rowe, and Michael Hirohama. The implementation of postgres. In *Making Databases Work: the Pragmatic Wisdom of Michael Stonebraker*, pages 519–559. Association for Computing Machinery and Morgan & Claypool, 2018.
- [44] Junaid Syed, Sultan Syed, and Bushra Aijaz. Reliability over unfettered autonomy: Advocating for deterministic orchestration in large language model tool integration. *Journal of Computer Science and Technology Studies*, 7(5):989–998, 2025.
- [45] Shiva Sai Krishna Anand Tokal, Vaibhav Jha, Anand Eswaran, Praveen Jayachandran, and Yogesh Simmhan. Towards orchestrating agentic applications as faas workflows. In *2025 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pages 1003–1010. IEEE, 2025.
- [46] Nidadavolu Venkat Durga Sai Siva Vara, Prasad Raju, Nuruzzaman Faruqui, Nikhil Patel, Olivia-Roxana Alecsiu, Priyabrata Thatoi, Salem A Alyami, and AKM Azad. Legalmind: Agentic ai-driven process optimization and cost reduction in legal services using deepseek. *IEEE Access*, 2025.
- [47] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [48] Padmanabham Venkiteela. The new interoperability paradigm model context protocol (mcp), apis, and the future of agentic ai. *Comput. Fraud Sec*, 8(1):1259–1271, 2025.

- [49] Padmanabhan Venkateela. n8n: An open-source workflow automation platform for enterprise integration and ai-driven orchestration. *International Journal of Computer Applications*, 975:8887, 2025.
- [50] Harsh Vishwakarma, Ankush Agarwal, Ojas Patil, Chaitanya Devaguptapu, and Mahesh Chandran. Can llms help you at work? a sandbox for evaluating llm agents in enterprise environments. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 9178–9212, 2025.
- [51] Vamsi Viswanadhapalli. The future of intelligent automation: how low-code/no-code platforms are transforming ai decisioning. *International Journal Of Engineering And Computer Science*, 14(1):26803–26825, 2025.
- [52] Hoang Vu, Nataliia Klievtsova, Henrik Leopold, Stefanie Rinderle-Ma, and Timotheus Kampik. Agentic business process management: Practitioner perspectives on agent governance in business processes. In *International Conference on Business Process Management*, pages 29–43. Springer, 2025.
- [53] Zhengxiang Wang, Jordan Kodner, and Owen Rambow. Exploring limitations of llm capabilities with multi-problem evaluation. In *The Sixth Workshop on Insights from Negative Results in NLP*, pages 121–140, 2025.
- [54] Zichong Wang, Zhibo Chu, Thang Viet Doan, Shiwen Ni, Min Yang, and Wenbin Zhang. History, development, and principles of large language models: an introductory survey. *AI and Ethics*, 5(3):1955–1971, 2025.
- [55] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversation. In *ICLR 2024 Workshop on Large Language Model (LLM) Agents*, 2024.
- [56] Mohan Yang, Nolan Lovett, Belle Li, and Zhen Hou. Towards dynamic learner state: Orchestrating ai agents and workplace performance via the model context protocol. *Education Sciences*, 15(8):1004, 2025.
- [57] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022.

- [58] Ruichen Zhang, Shunpu Tang, Yinqiu Liu, Dusit Niyato, Zehui Xiong, Sumei Sun, Shiwen Mao, and Zhu Han. Toward agentic ai: generative information retrieval inspired intelligent communications and networking. *IEEE Communications Magazine*, 64(1):197–204, 2025.
- [59] Yutao Zhu, Huaying Yuan, Shuting Wang, Jiongnan Liu, Wenhan Liu, Chenlong Deng, Haonan Chen, Zheng Liu, Zhicheng Dou, and Ji-Rong Wen. Large language models for information retrieval: A survey. *ACM Transactions on Information Systems*, 2023.